

数据库系统概论 & 前沿技术与应用

SJTU 周煊赫

助教：任子谦

zhouxh@cs.sjtu.edu.cn

<https://db.zhouxh.store>



■ 上海交通大学计算机系，助理教授

■ 研究方向：数据系统、文档智能、统一世界模型

■ 研究情况：

- ✓ 发表数十篇国际顶会/顶刊系统级论文，引用三千余次；
- ✓ 多次入选2020-2024年 NIPS、VLDB、ICDE 顶会高被引用论文；
- ✓ 多篇论文选入知名高校课程（如CMU 15-799、Cornell 7390）

■ 代表项目：

- ✓ **openGauss** 自治版内核，3000+星标，下载量破300万次，通信学会科学技术一等奖
- ✓ **DB-GPT** 自治运维 (Benchcouncil Top100 开源成就)、**BMTools** (2.9k star)
- ✓ **Bird-SQL**，被openAI采用推广其微调服务

■ 获奖情况：

- ✓ VLDB 2023 最佳工业论文奖、SIGMOD Jim Gray提名奖（大陆首次）
- ✓ 微软学者、字节跳动奖学金、清华特奖



目录

- 商业数据库之父发家史
- 数据库系统概述
- 数据库发展简史
- 数据库系统的应用和展望
- 本章小结

目录

- 商业数据库之父发家史
- 数据库系统概述
- 数据库发展简史
- 数据库系统的应用和展望
- 本章小结

商业数据库之父——拉里·埃里森



商业数据库之父——拉里·埃里森



商业数据库之父——拉里·埃里森

1962 in 伊利诺斯大学

早年在IBM

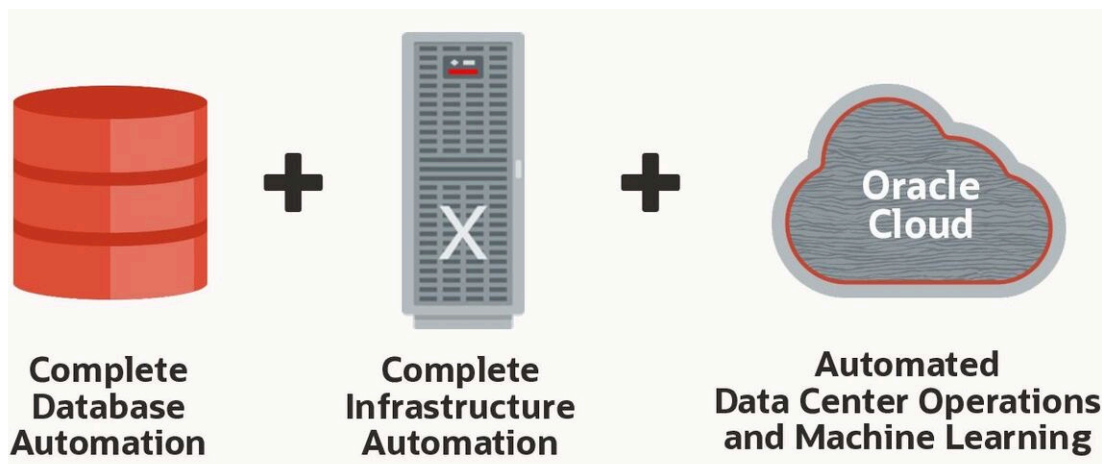
第一桶金

70年代，数据库技术已经广泛应用于商场，成千上万种商品的品牌、价格、生产日期等复杂信息都通过数据库存入电脑，每一种商品都被编码标号，出纳员只要用扫描机识别这一号码，就可以快速打印出商品的名称、数量、价格等信息，大大节省了人力和时间。正是数据库技术使超级市场变成了今天的模样。

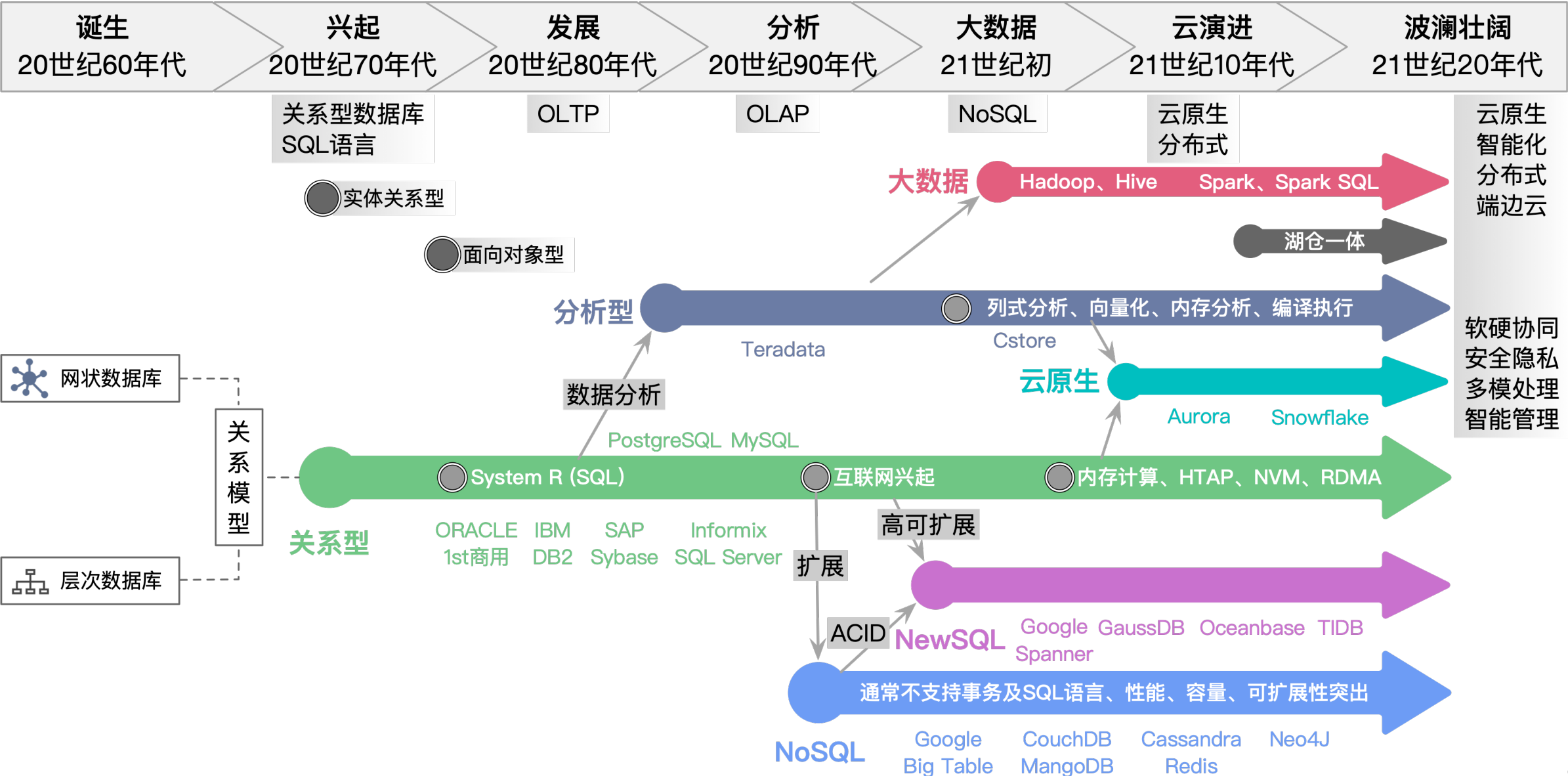
找到对手：微软

数据库 + AI，开启新时代

“无趣”的亿万富翁生活



数据库发展史概览

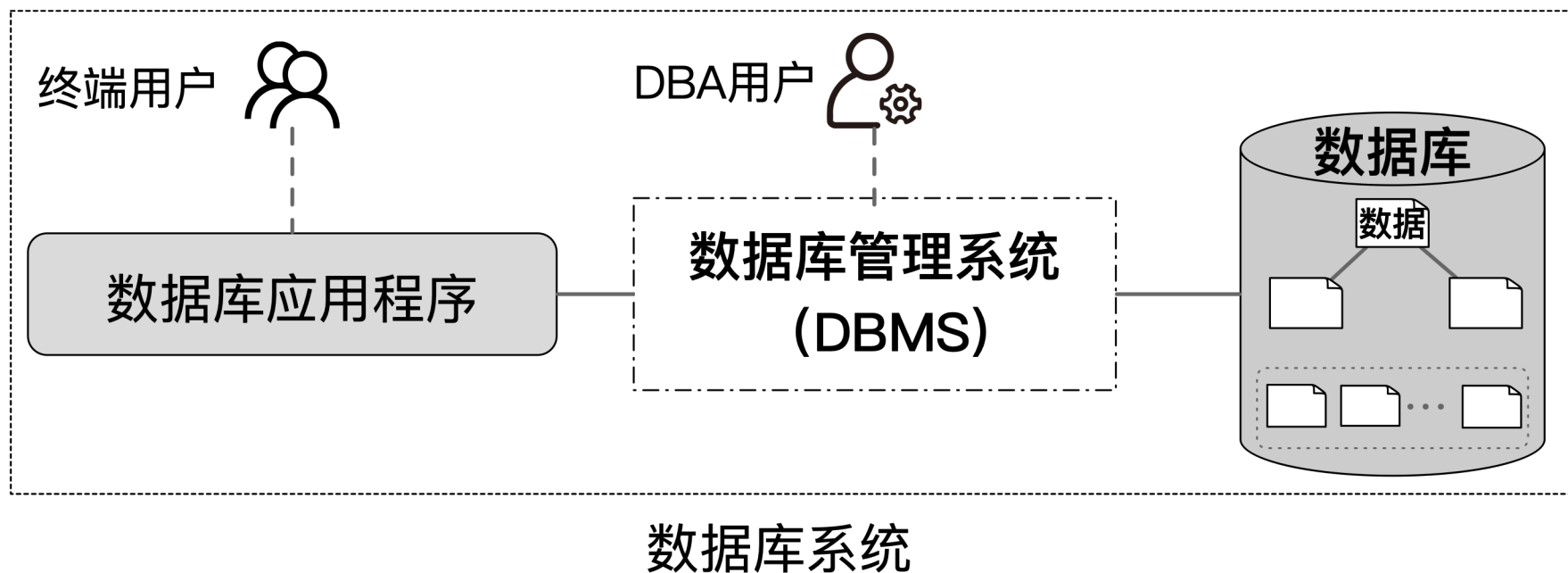


目录

- 商业数据库之父发家史
- 数据库系统概述
- 数据库发展简史
- 数据库系统的应用和展望
- 本章小结

数据库的基本概念

- 数据 (Data)
- 数据库 (Database, DB)
- 数据库管理系统 (Database Management System, DBMS)
- 数据库系统 (Database System, DBS)



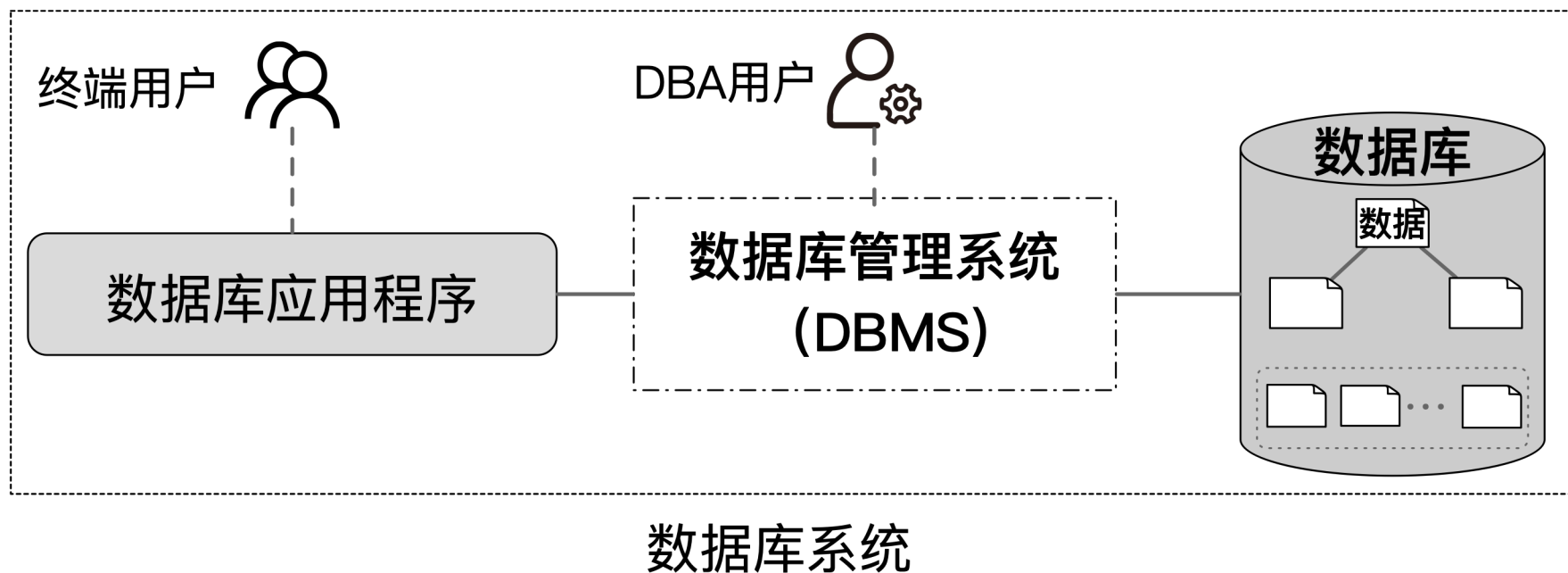
数据库的基本概念

□数据（Data）

□数据库（Database, DB）

□数据库管理系统（Database Management System, DBMS）

□数据库系统（Database System, DBS）



数据

□什么是数据？

- 在现实生活，数据是可识别的抽象符号，即描述事物的符号记录。
- 在计算机中，数据是所有能被计算机处理的符号的总称
- 在数据库中，数据是数据库中存储的基本对象

□数据的种类：

- 数字，例如 1、2、3
- 文本
- 图像
- 音频
- ...

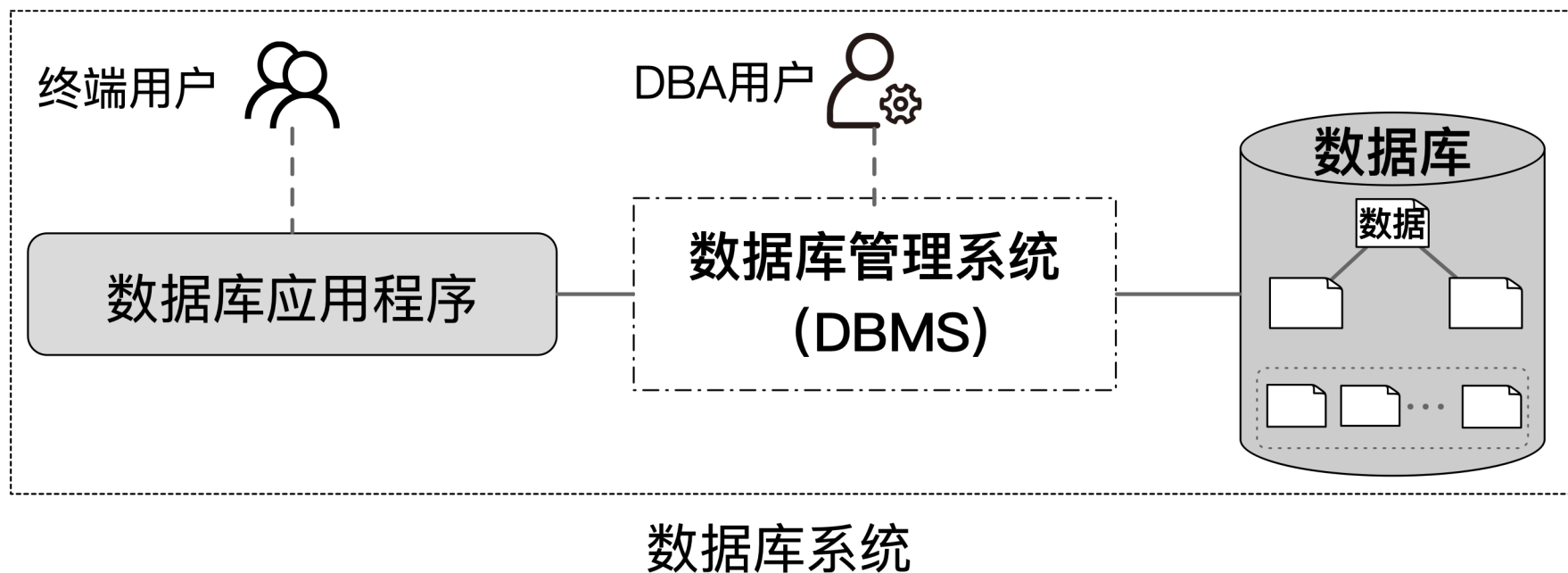
数据库的基本概念

□ 数据 (Data)

□ 数据库 (Database, DB)

□ 数据库管理系统 (Database Management System, DBMS)

□ 数据库系统 (Database System, DBS)



数据库

□什么是数据库？

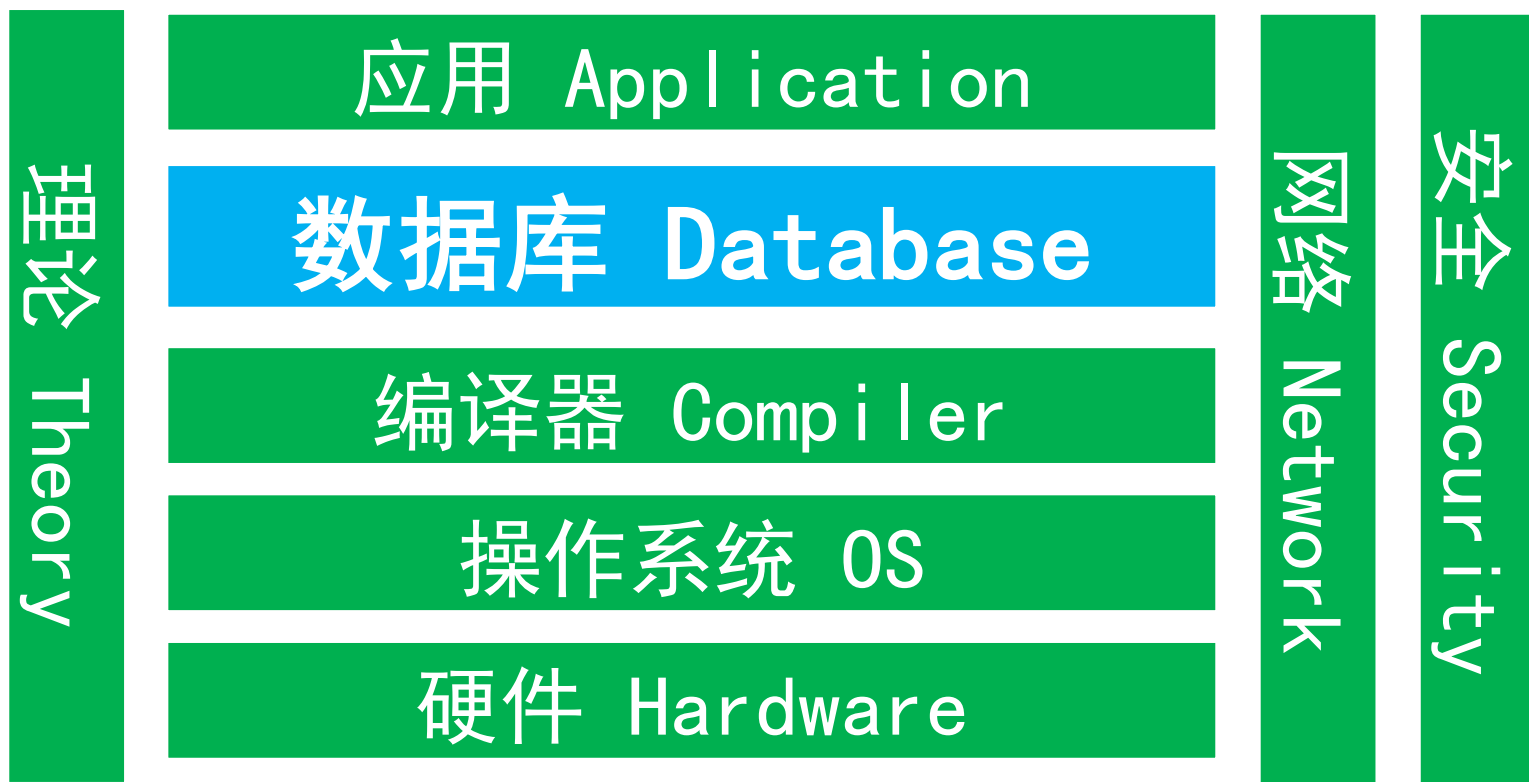
- 定义1：是长期储存在计算机中的有组织的、可管理和可共享的数据集合
- 定义2：就是指一组相互有关联的数据集合

□数据库的基本特征

- 数据按一定的数据模型组织、描述和储存
- 可为各种用户共享
- 冗余度较小
- 数据独立性较高
- 易扩展

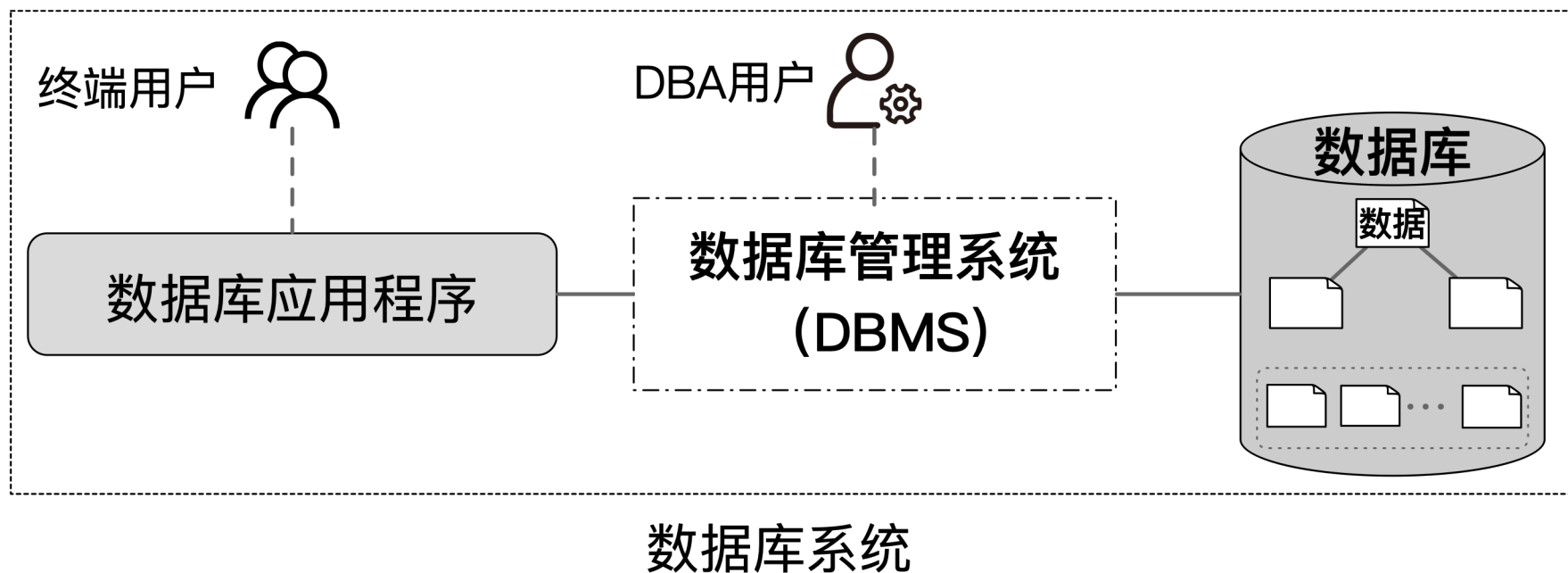
数据库在计算机系统的位置

数据库是计算机的核心基础软件之一



数据库的基本概念

- 数据 (Data)
- 数据库 (Database, DB)
- 数据库管理系统 (Database Management System, DBMS)
- 数据库系统 (Database System, DBS)



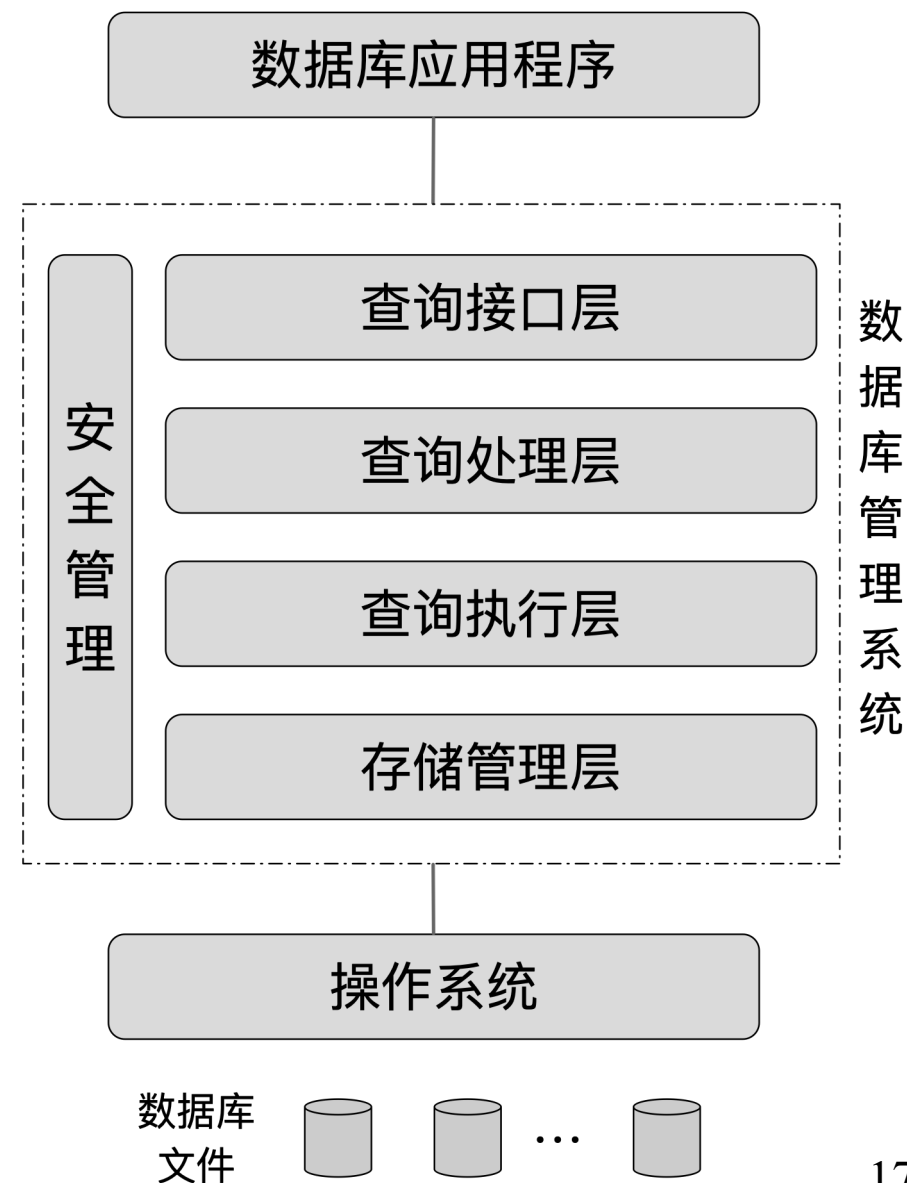
数据库管理系统

□什么是数据库管理系统？

- 定义1：是用户（应用程序）与操作系统之间的**数据管理软件**
- 定义2：是一个大型复杂的基础软件系统

□数据库管理系统的用途

- 科学地组织和存储数据
- 高效地获取和管理数据
- 负责对数据库进行统一的管理和控制
- 保证数据库的可用性和安全性
- 为用户提供进行数据存取和管理的接口



数据库管理系统的主要功能

□ 数据定义功能

- 提供数据定义语言（data definition language, DDL）
- 定义数据库中的数据对象

□ 数据组织、存储和管理

- 分类组织、存储和管理各种数据
- 确定组织数据的文件结构和存取方式
- 实现数据之间的联系
- 提供多种存取方法提高存取效率

数据库管理系统的主要功能（续）

□数据操纵功能

- 提供数据操纵语言（data manipulation language, DML）
- 实现对数据库的基本操作（查询、插入、删除和修改）

□数据库的事务管理和运行管理

- 数据库在建立、运行和维护时由数据库管理系统统一管理和控制
- 保证数据的安全性、完整性、多用户对数据的并发使用
- 发生故障后的系统恢复

数据库管理系统的主要功能（续）

□数据库的建立和维护功能

- 数据库初始数据的装载和转换
- 数据库转储、恢复功能
- 数据库的重组织
- 性能监视、分析等

□其它功能

- 数据库管理系统与网络中其它软件系统的通信
- 数据库管理系统系统之间的数据转换
- 异构数据库之间的互访和互操作

数据库管理系统概述

□ 查询接口层

- 负责用户和数据库管理系统的交互
- SQL 查询语言

□ 查询处理层

- 对数据库的各类语句进行词法分析等
- 授权检验、视图转换和查询优化等任务

□ 查询执行层

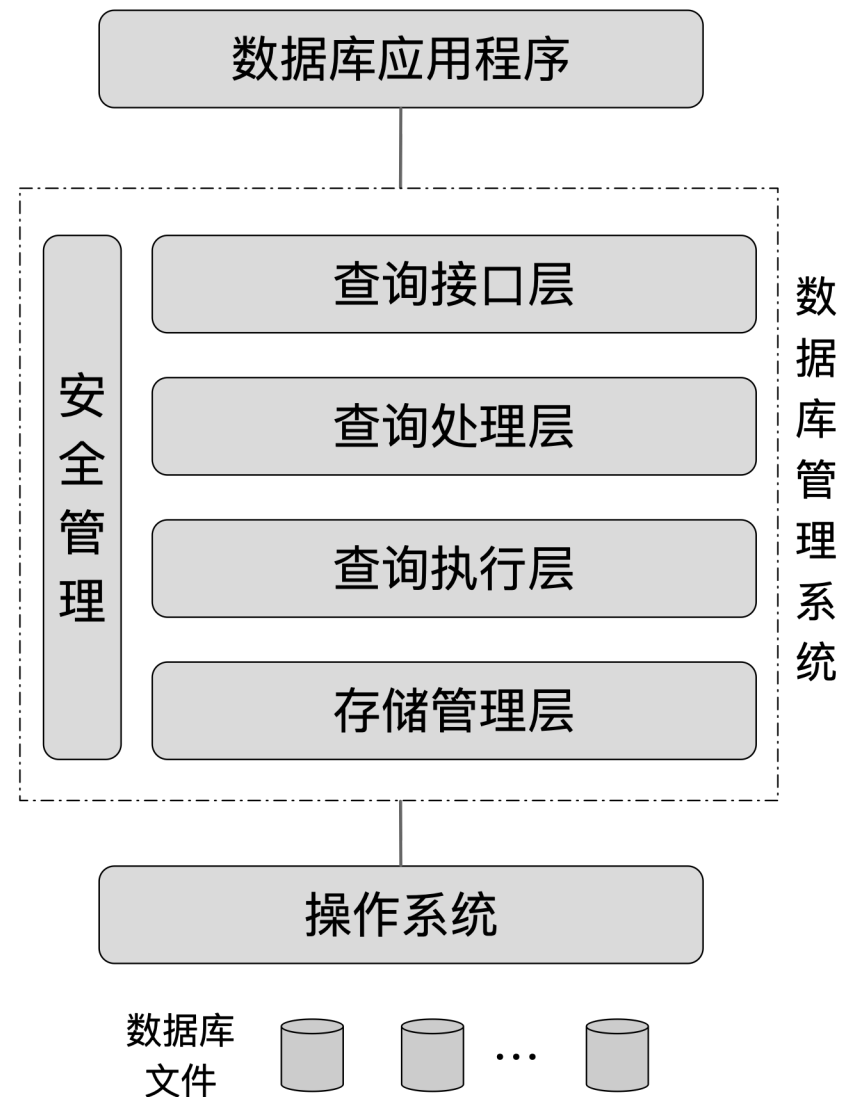
- 形成数据库语言语句的执行计划
- 完成数据存取、路径维护和并发控制等

□ 存储管理层

- 在数据存储器中完成数据的增删改查操作

□ 安全管理

- 负责数据库全生命周期的安全管理



数据库管理系统

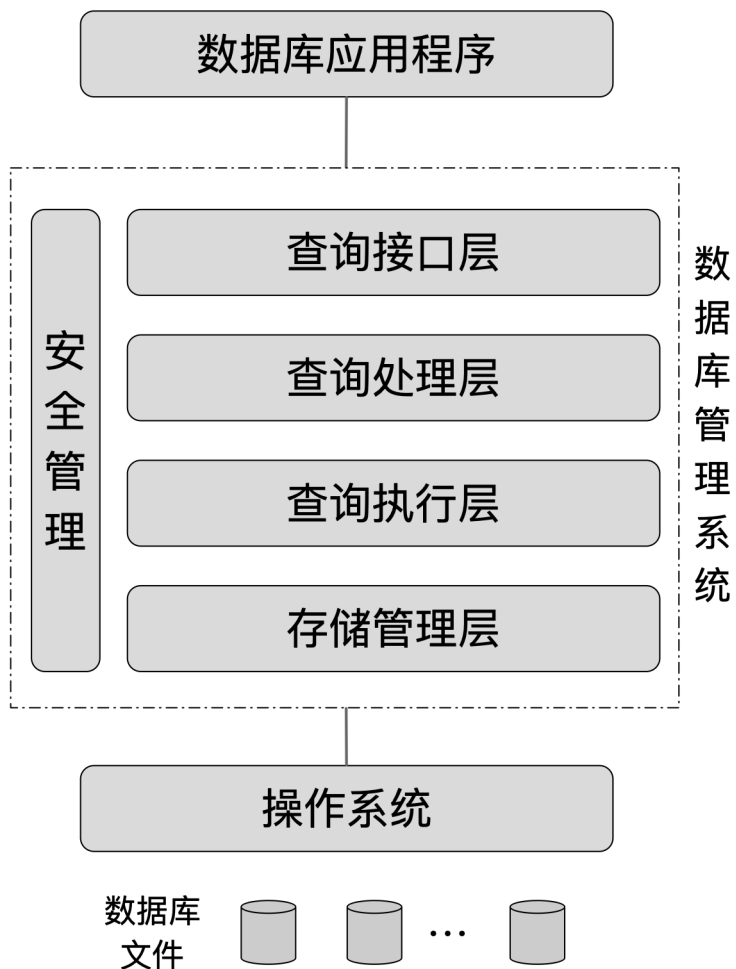
数据库管理系统概述

□ 查询接口层

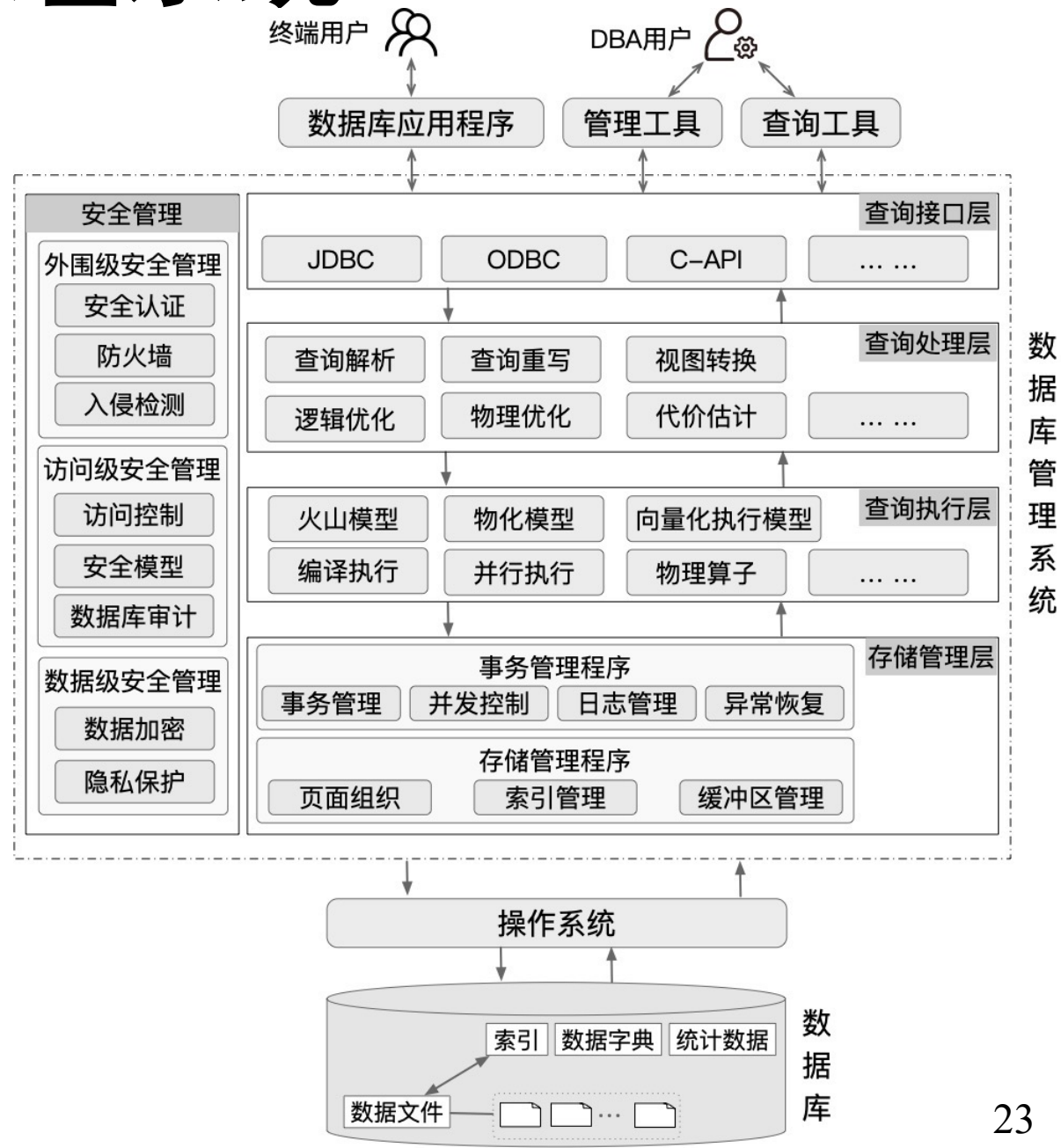
Type	Source	Target	SQLGlot	jOOQ	SQLines	GPT-4o	CrackSQL	Dialect
S1	1 / col	1 / NULLIF(col, 0)	✓	×	×	×	✓	MySQL → PG
S2	tbl AS tbl_alias	tbl tbl_alias	✓	✓	×	✓	✓	PG → Oracle
S3	FROM sub_query	FROM sub_query AS query_alias	×	✓	✓	×	✓	Oracle → MySQL
S4	HAVING ... GROUP BY ...	GROUP BY ... HAVING ...	✓	✓	×	✓	✓	Oracle → PG
K1	GROUP BY ROLLUP(col_list)	GROUP BY col_list WITH ROLLUP	×	✓	×	×	✓	PG → MySQL
K2	... tbl1 FULL OUTER JOIN tbl2 ON ...	... tbl1 LEFT OUTER JOIN tbl2 ON ... UNION ALL ... tbl1 RIGHT OUTER JOIN tbl2 ON ... WHERE NOT EXISTS (SELECT 1 FROM tbl1 WHERE ...)	✓	×	×	×	✓	PG → MySQL
K3	LIMIT num1, num2	LIMIT num2 OFFSET num1	✓	✓	×	✓	✓	MySQL → PG
K4	WHERE ROWNUM <= num	LIMIT num	×	✓	×	×	✓	Oracle → PG
K5	FROM DUAL	/	×	✓	✓	×	✓	Oracle → PG
F1	EXTRACT(unit FROM AGE(time1, time2))	TIMESTAMPDIFF(unit, time2, time1)	×	×	×	×	✓	PG → MySQL
F2	str1 ILIKE str2	LOWER(str1) LIKE LOWER(str2)	✓	✓	×	×	✓	PG → MySQL
F3	TO_TIMESTAMP(unix_time)	FROM_UNIXTIME(unix_time)	✓	×	×	✓	✓	PG → MySQL
F4	CURDATE()	CURRENT_DATE	×	✓	×	✓	✓	MySQL → PG
F5	LAST_DAY(time)	DATE_TRUNC('MONTH', time) + INTERVAL '1 MONTH' - INTERVAL '1 DAY'	✓	×	×	✓	✓	MySQL → PG
D1	CAST(col AS bytea)	CAST(col AS BINARY)	×	×	×	✓	✓	PG → MySQL
D2	DATE_FORMAT(time1, unit_format) - DATE_FORMAT(time2, unit_format)	EXTRACT(unit FROM time1) - EXTRACT(unit FROM time2)	×	×	×	×	✓	MySQL → PG
D3	date_obj = time	date_obj = TO_DATE(time, format)	×	×	×	✓	✓	MySQL → Oracle
D4	DATETIME	TIMESTAMP	×	✓	✓	✓	✓	MySQL → Oracle
D5	SUM(bool_expr)	SUM(CASE WHEN bool_expr THEN 1 ELSE 0 END)	×	✓	✓	✓	✓	MySQL → Oracle

- 负责数据库全生命周期的安全管理

数据库管理系统



具体化
→



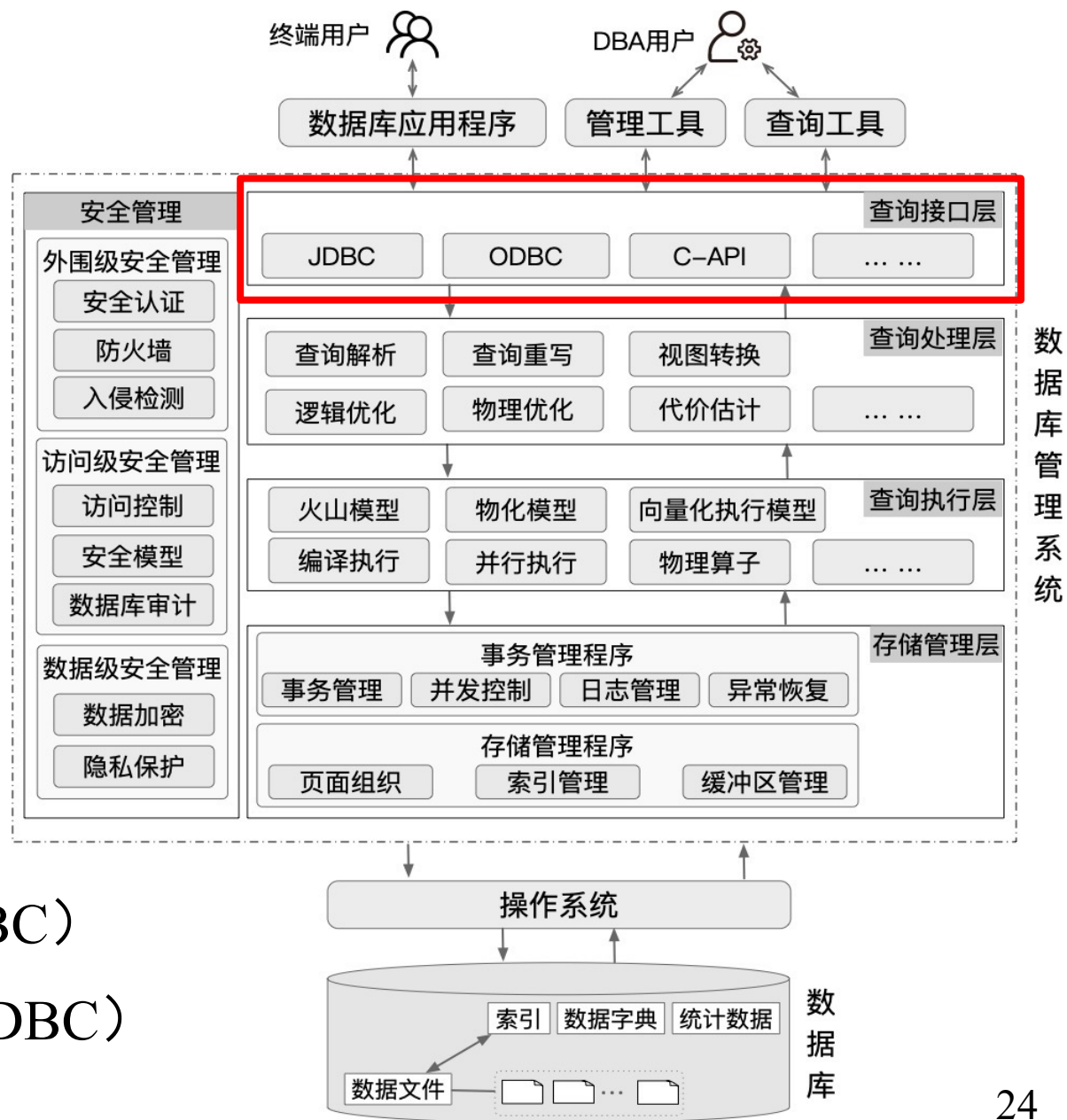
查询接口层

□查询接口层

- 负责用户和数据库管理系统的交互
- SQL查询语言

□常见的查询接口

- JDBC (Java database connectivity, JDBC)
- ODBC (open database connectivity, ODBC)



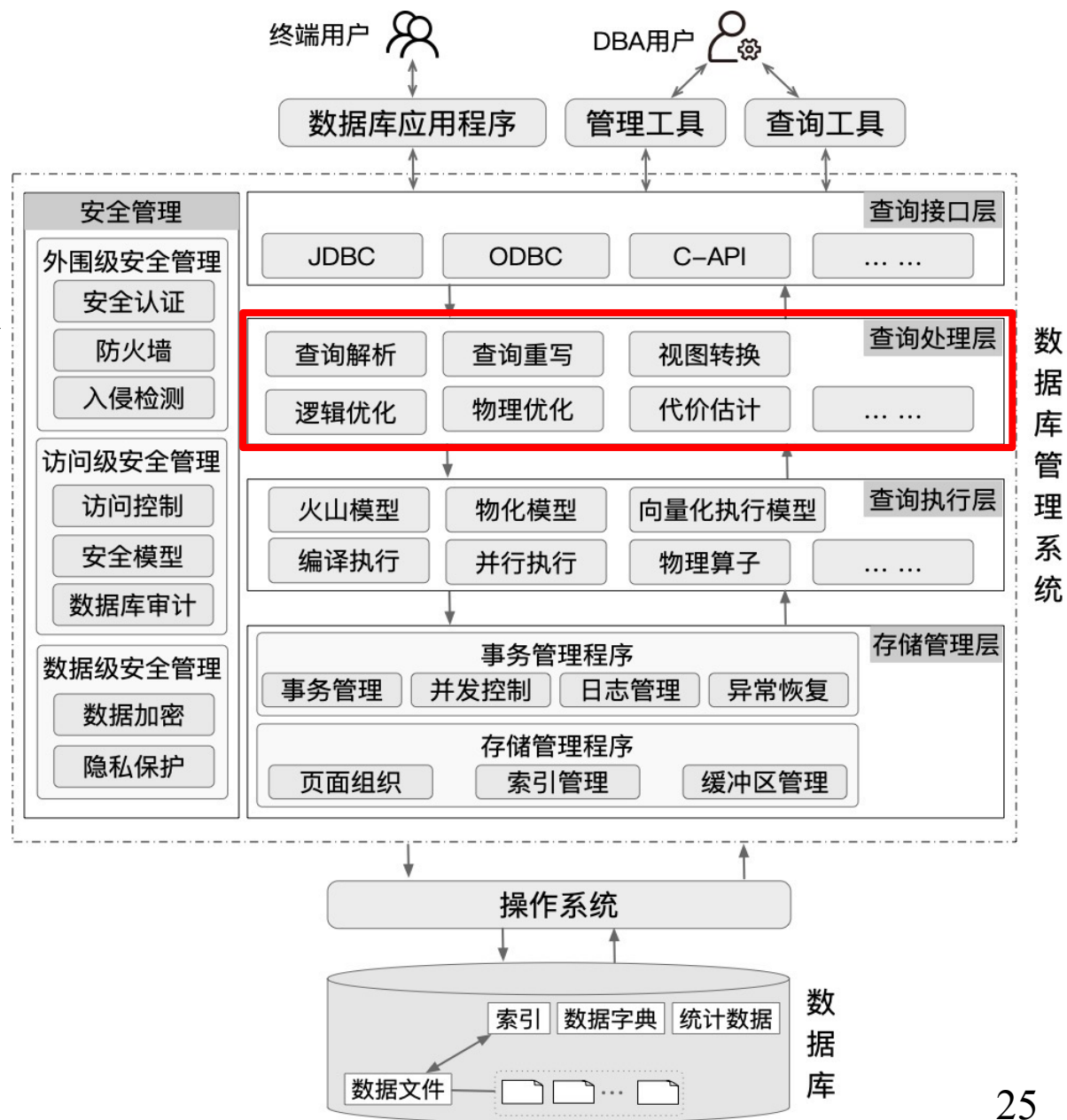
查询处理层

□ 查询处理层

- 对数据库的各类语句进行词法分析等
- 授权检验、视图转换和查询优化等任务

□ 查询处理层组成

- 查询解析
- 查询重写
- 视图转换
- 逻辑优化
- 物理优化
- 代价估计



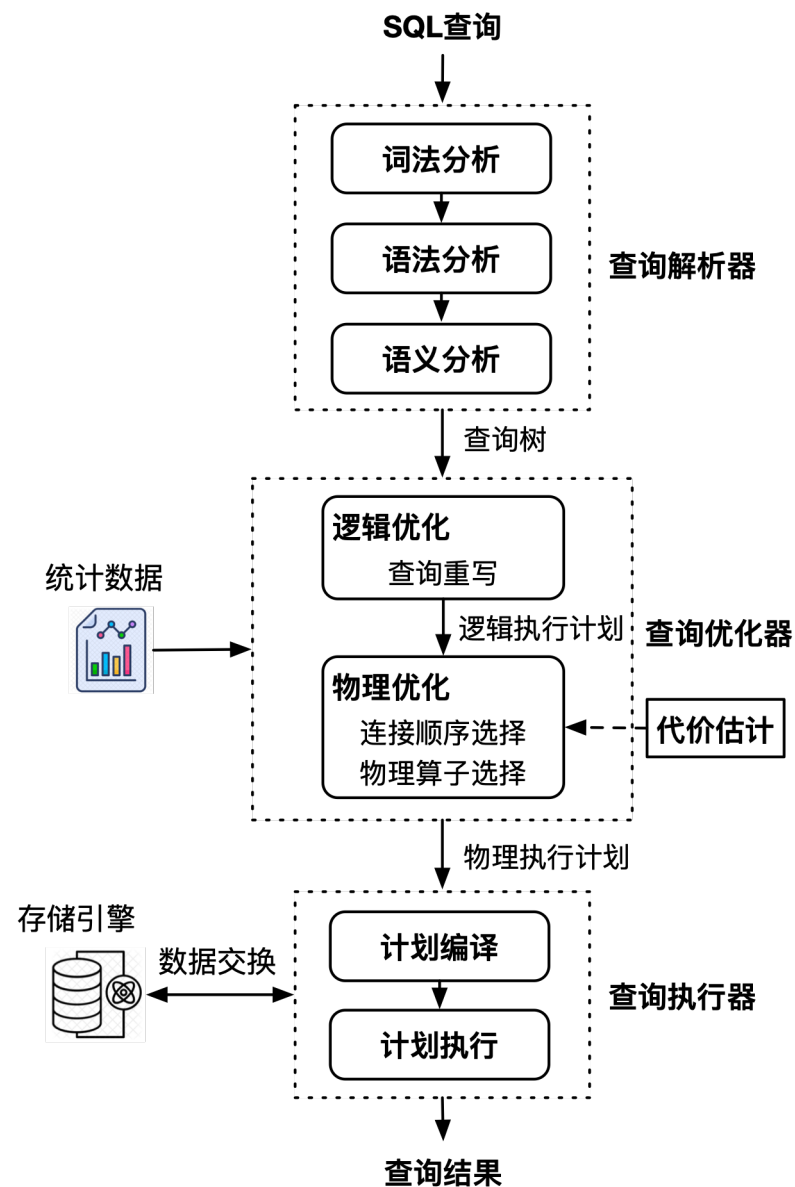
查询处理层（续）

□ 查询处理是指针对一条查询，从数据库中提取目标数据时对该查询进行的处理过程，包括：

□ 查询解析：将查询语句转化为容易被数据库执行的表达

□ 查询优化：为优化查询的执行计划而进行各种逻辑等价的转换

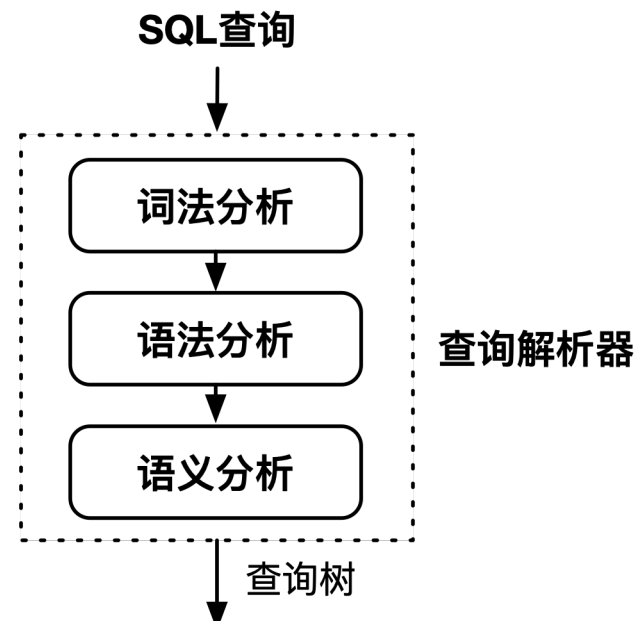
□ 查询执行：查询的实际执行



查询处理层（续）

□ 查询解析

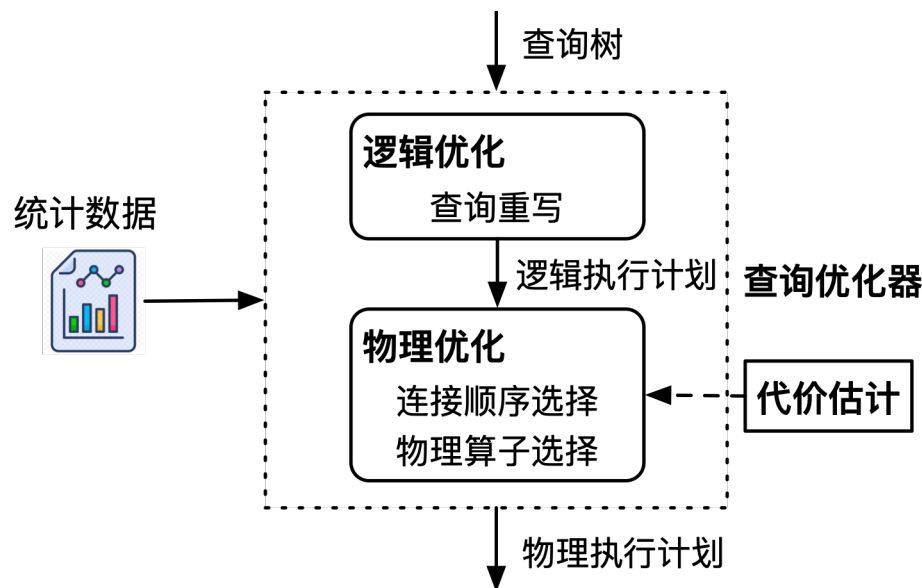
- 输入SQL查询，输出查询树
- 词法分析：提取出SQL查询中的关键词、常量等成分
- 语法分析：将词法分析提取的成分构建为一颗语法分析树
- 语义分析：检查语法分析树的语义正确性，并将其转化为基于关系代数式表达的查询树



查询处理层（续）

□查询优化

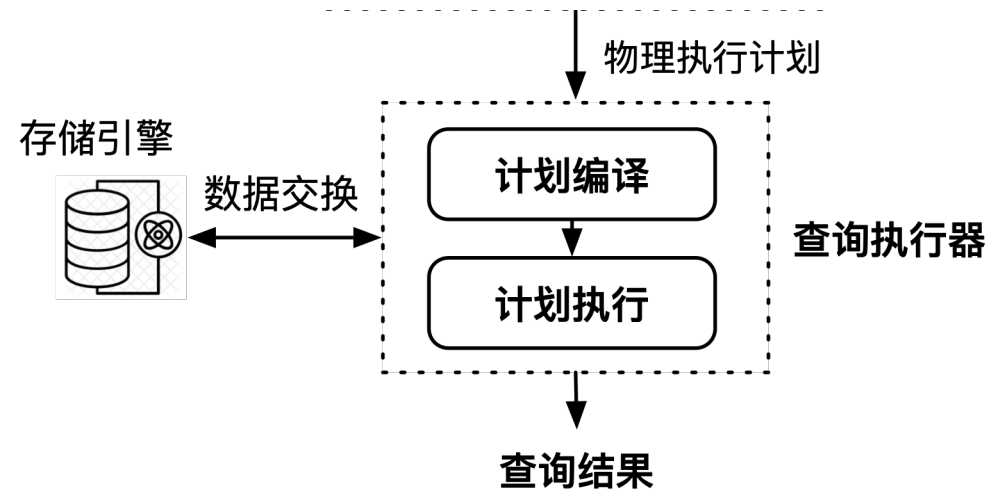
- 输入查询树，输出物理执行计划
- 逻辑优化：依赖于逻辑等价变换规则将关系代数表达式变换为有更高执行效率的形式
- 代价估计：用来预测各执行计划的资源消耗大小
- 物理优化：依赖于代价估计器估计各执行计划的代价，以从中选取资源消耗最小的执行计划



查询处理层（续）

□ 查询执行

- 输入为查询的执行计划，输出为查询结果
- 计划编译：将查询的执行计划编译可执行的编程代码，甚至是机器码
- 计划执行：执行计划编译的结果，与使用何种查询执行模型有关



存储管理层

□ 存储管理层

- 根据查询执行层的操作指令，在数据存储器中完成数据的增删改查操作

□ 存储管理层组成

- 事务管理程序
 - 事务管理
 - 并发控制
 - 异常恢复
- 存储管理程序
 - 存储管理
 - 索引管理



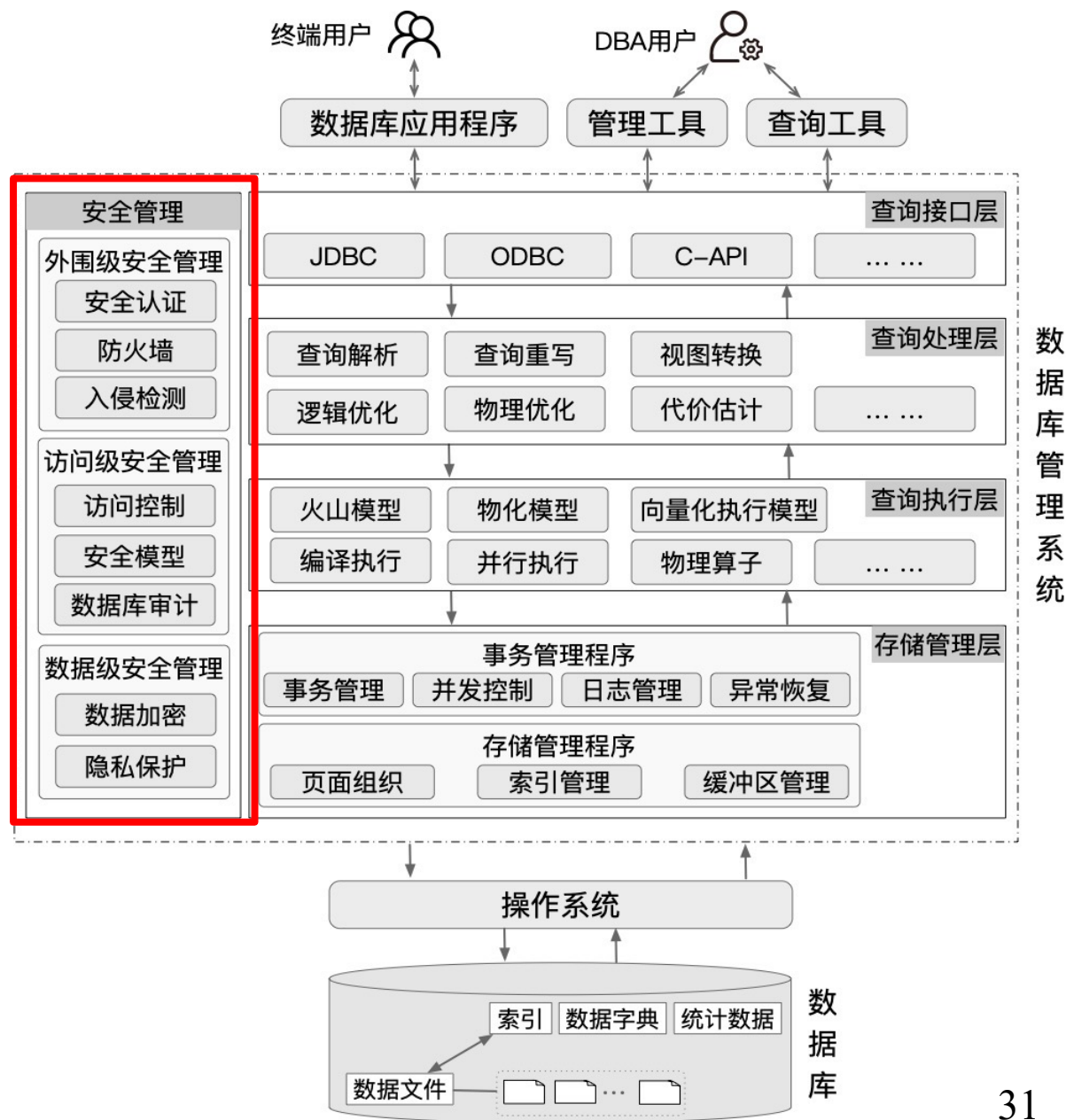
安全管理层

安全管理层

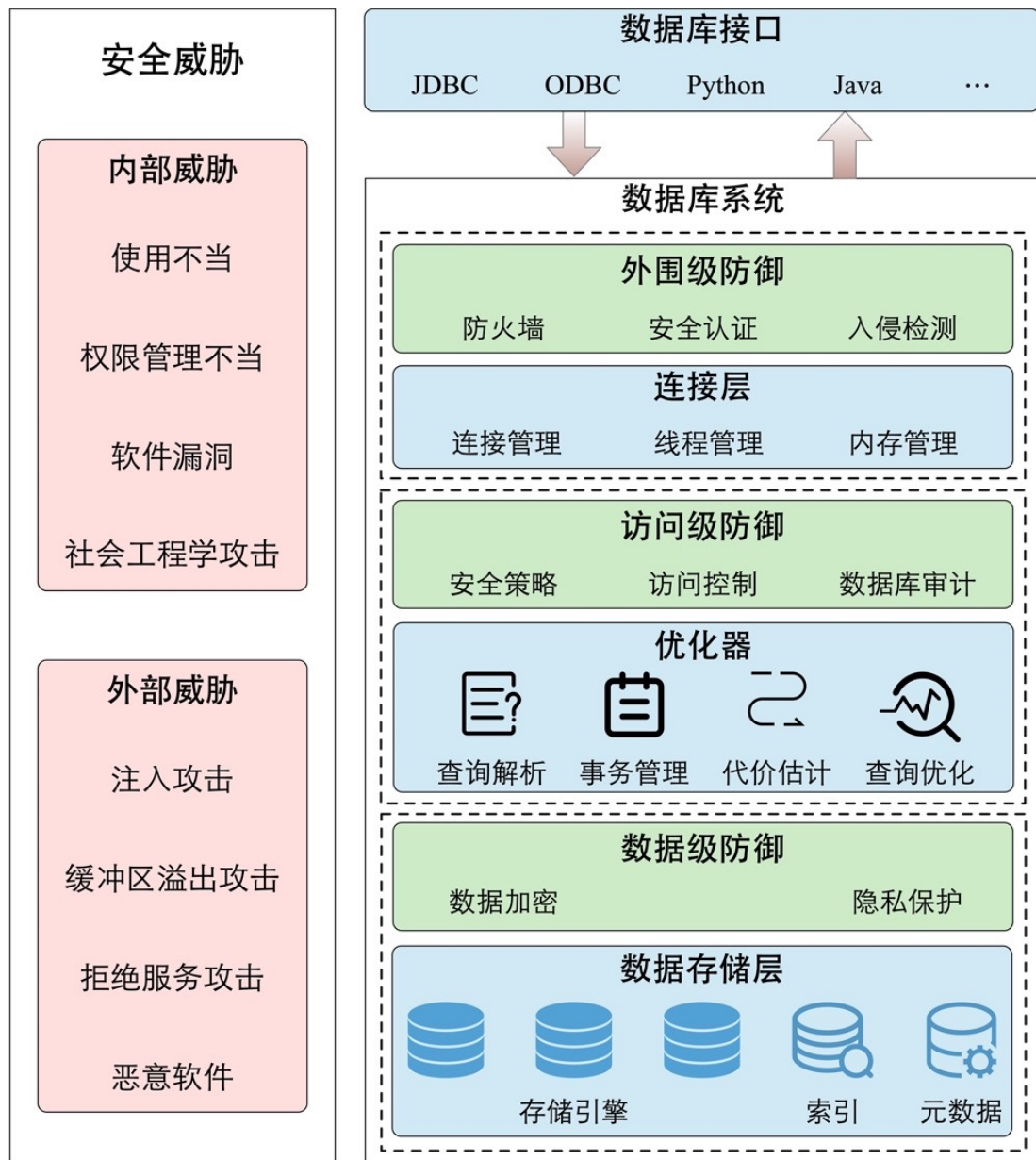
- 根据查询执行层的操作指令，在数据存储器中完成数据的增删改查操作

安全管理层组成

- 数据库安全
 - 外围级安全管理
 - 访问级安全管理
 - 数据级安全管理
- 数据库高可用



数据库安全简介



□ 外围级防御

- 决定哪些用户可以访问数据库
- 安全认证、防火墙、入侵检测

□ 访问级防御

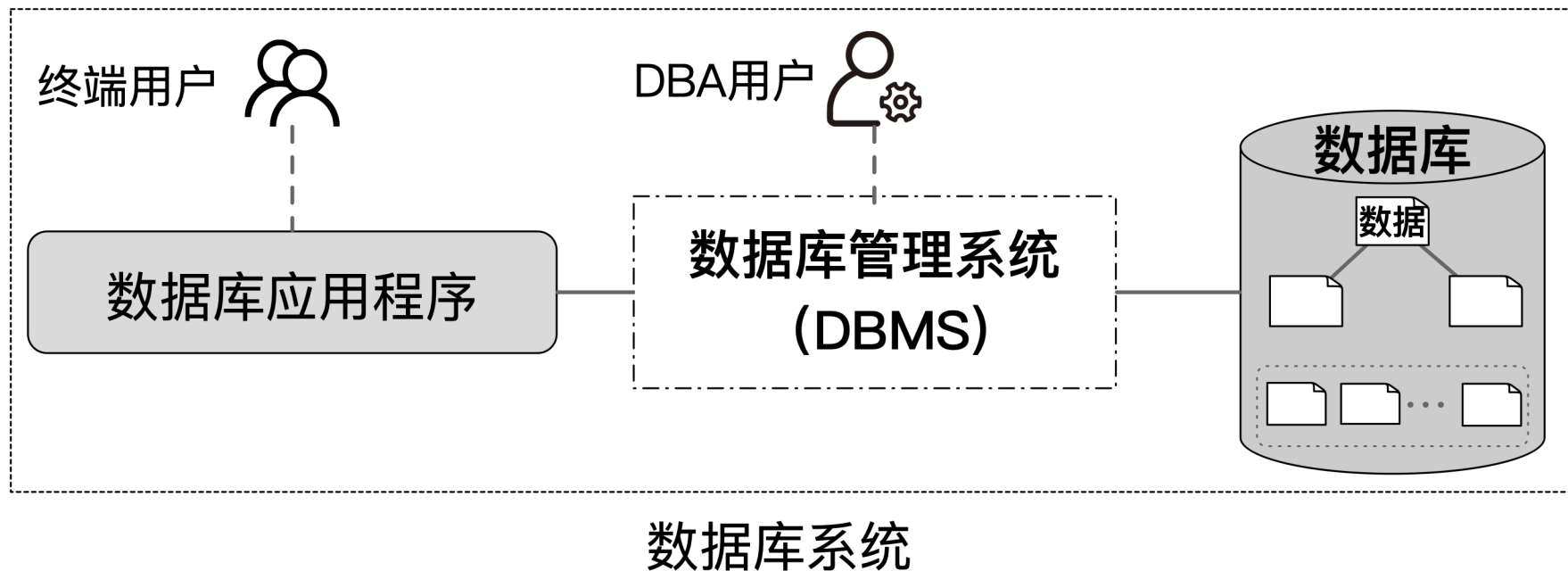
- 控制允许哪些用户有权访问某些数据或包含它的系统
- 安全配置策略、访问控制、审计

□ 数据级防御

- 预防和应对发生在数据库本身的安全隐患
- 数据加密、隐私保护

数据库的基本概念

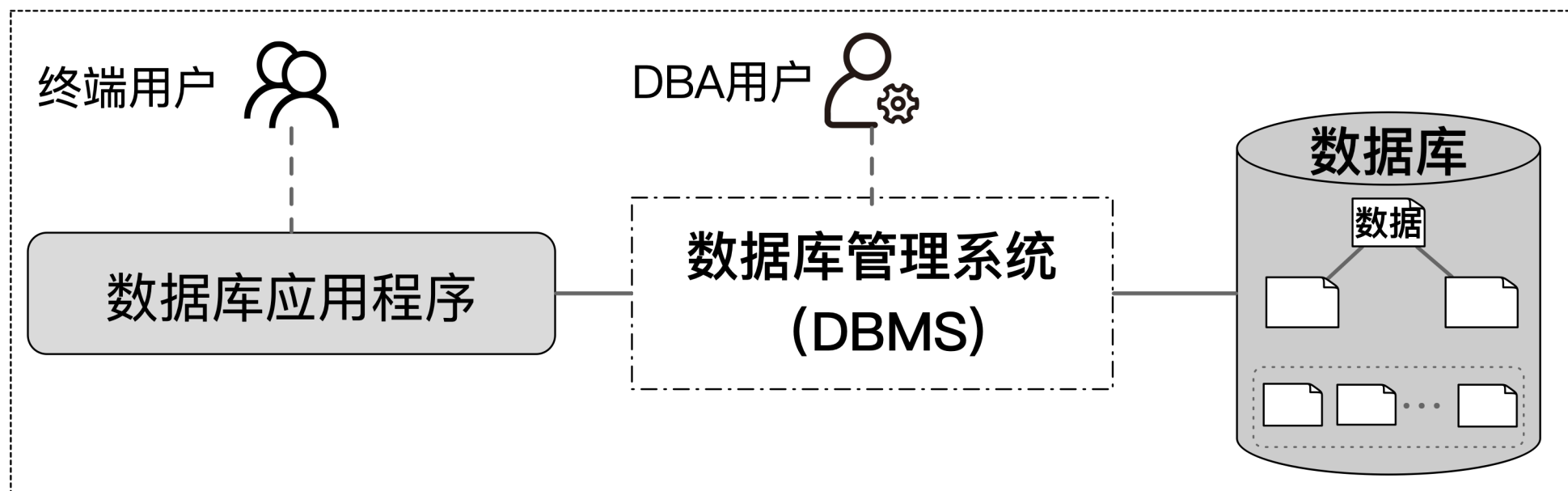
- 数据 (Data)
- 数据库 (Database, DB)
- 数据库管理系统 (Database Management System, DBMS)
- 数据库系统 (Database System, DBS)



数据库系统

□什么是数据库系统？

- 数据库系统是指在计算机系统中引入数据库之后的系统，一般由数据库、数据库管理系统及其开发工具、应用系统、数据库管理员和终端用户构成。口语中的常说的“数据库系统”在大多数时候都是指DBMS。



数据库系统

目录

- 商业数据库之父发家史
- 数据库系统概述
- 数据库发展简史
- 数据库系统的应用和展望
- 本章小结

数据管理的三个阶段

— 20世纪50年代中期

20世纪50年代中期 — 60年代中期

20世纪60年代末 — 至今

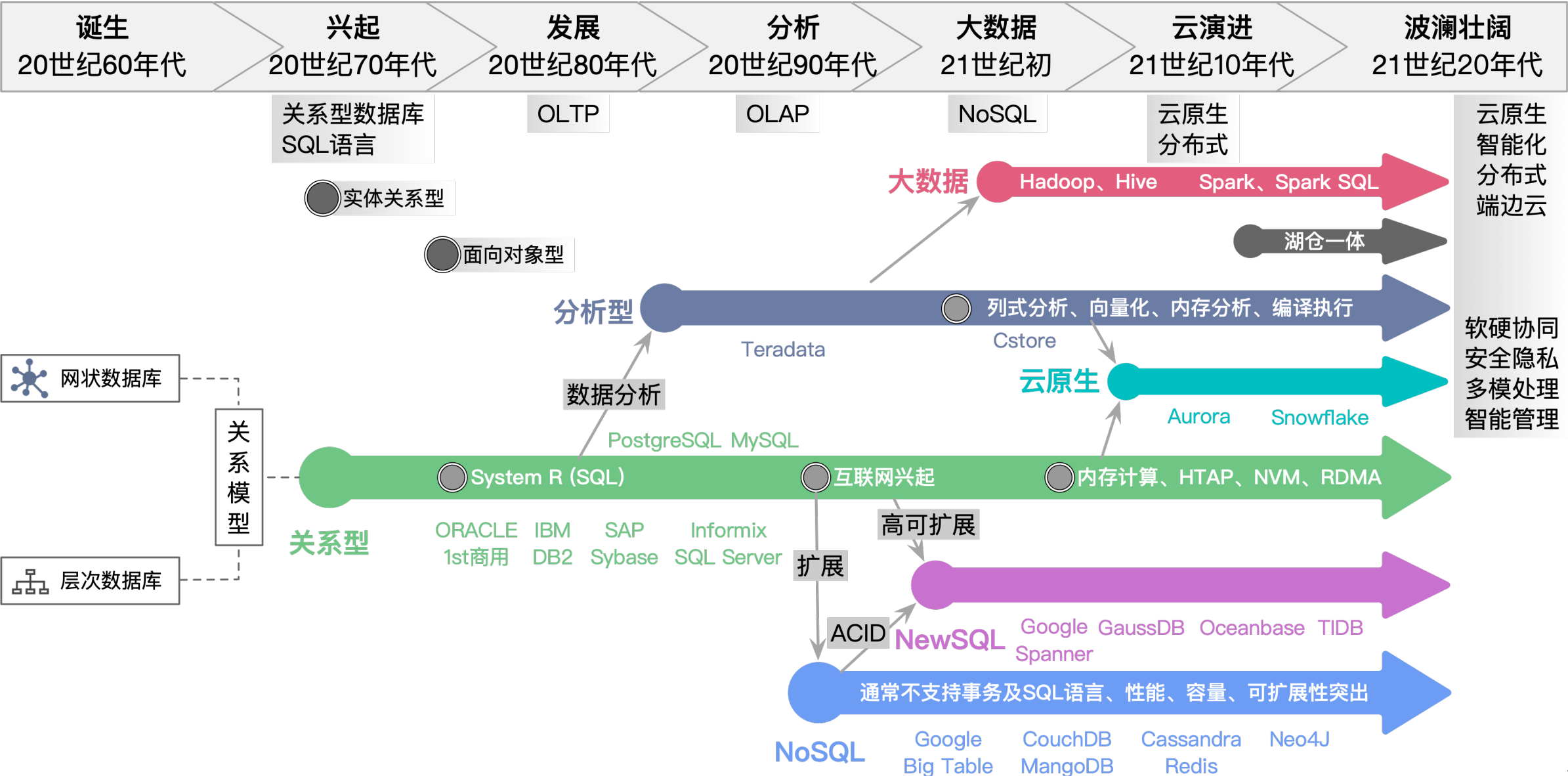
人工管理

文件系统管理

数据库管理

	人工管理	文件系统管理	数据库管理
应用背景	满足科学计算	满足科学计算和数据管理	大规模数据管理
硬件条件	无直接存取存储设备	磁盘、磁鼓	大容量磁盘
软件条件	没有操作系统	有文件系统	数据库管理系统
处理方式	批处理	联机实时处理、批处理	联机实时处理、批处理、分布式处理
实现方式	通过卡带、卡片和磁带等计算机间接存取设备实现	通过磁盘和磁鼓等计算机直接存取设备实现	通过数据库管理系统与数据库进行交互以实现数据管理操作
特点	- 人力维护成本极高 - 数据与应用程序深度绑定 - 数据间缺乏组织和独立性 - 数据无共享、冗余度极大	- 人力维护成本很高 - 数据文件与应用程序耦合 - 数据共享性差、冗余度大 - 不支持多用户并发访问数据	- 数据使用数据模型描述，无需应用程序定义，组织程度和共享程度高 - 编程友好，无需关心数据库文件的物理操作和系统控制 - 支持多用户并发访问

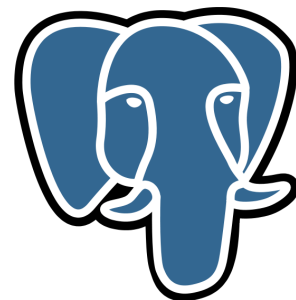
数据库发展史概览



数据库发展史概览

- 层次和网状数据库
- 关系数据库
- OLTP、OLAP与HTAP数据库
- NoSQL数据库
- 分布式数据库
- 内存数据库
- 云数据库
- 其它类型数据库
 - 实时数据库
 - 并行数据库
 - 终端数据库

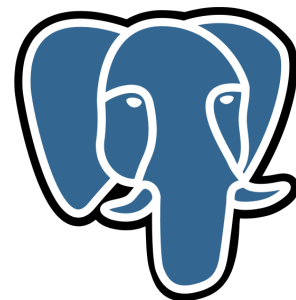
ORACLE®
DATABASE



数据库发展史概览

- 层次和网状数据库
- 关系数据库
- OLTP、OLAP与HTAP数据库
- NoSQL数据库
- 分布式数据库
- 内存数据库
- 云数据库
- 其它类型数据库
 - 实时数据库
 - 并行数据库
 - 终端数据库

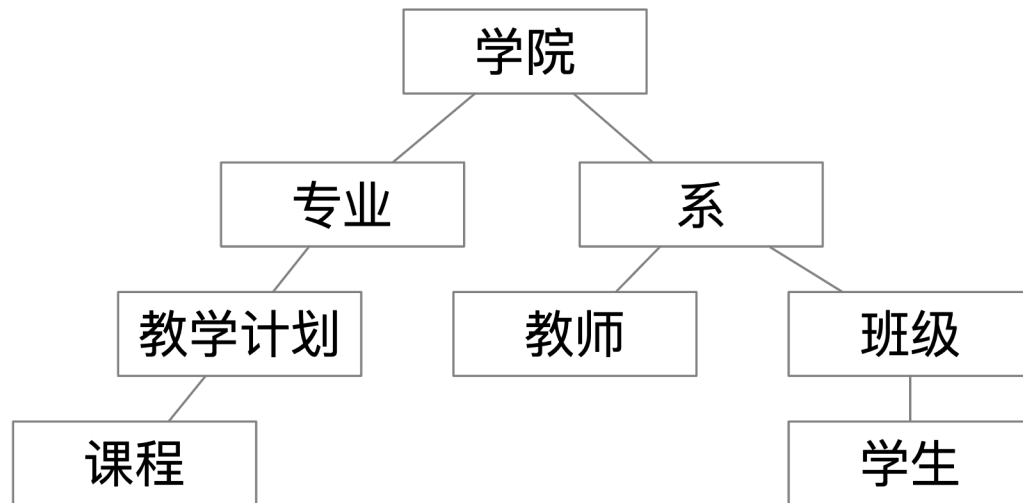
ORACLE®
DATABASE



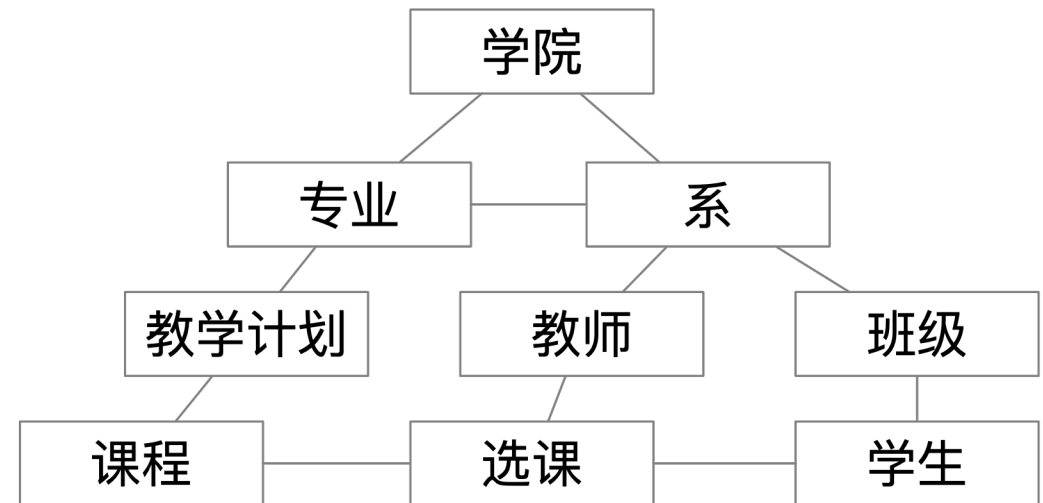
层次数据库和网状数据库

□在数据库发展的早期，有两类重要的数据库

- 层次数据库（Hierarchical Database）和网状数据库（Network Database）
- 层次数据库基于层次模型，以“树结构”来表示数据记录之间的联系
- 网状数据库基于网状模型，以“图结构”来表示数据记录之间的联系



(1) 层次数据模型



(2) 网状数据模型

层次模型与层次数据库

- 层次模型是数据库系统中最早出现的数据模型
- 层次模型用树形结构来表示各类实体以及实体间的联系
- 层次数据库系统的典型代表是IBM公司的IMS（Information Management System, IMS）数据库
- 层次模型
 - 有且只有一个结点没有双亲结点，这个结点是根结点
 - 除根节点以外，其他结点有且只有一个双亲结点

网状模型和网状数据库

- 网状模型是网状数据库的组织方式
- 1964年，美国通用电气公司发布了世界上第一个网状数据库系统——IDS（Integrated Data Storage，集成数据存储）
- 网状模型：
 - 允许一个以上的结点无双亲
 - 一个结点可以有多个的双亲

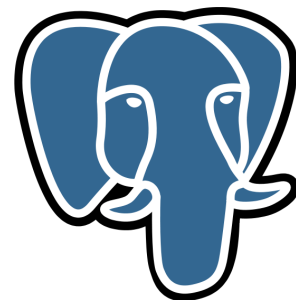


查理士·巴赫曼（Charles W. Bachman）
1973年获得图灵奖，以表彰他在数据库领域，
尤其是网状数据库系统方面的杰出贡献。42

数据库发展史概览

- 层次和网状数据库
- 关系数据库
- OLTP、OLAP与HTAP数据库
- NoSQL数据库
- 分布式数据库
- 内存数据库
- 云数据库
- 其它类型数据库
 - 实时数据库
 - 并行数据库
 - 终端数据库

ORACLE®
DATABASE



关系数据库

□关系数据库

– 关系数据库是表的集合：

- 每个表有唯一的名字
- 每个表可以有多个列
- 每个列有唯一的名字
- 表中的一行代表的是一系列值之间的联系，即表是关系的集合

– 表的概念和数学上关系的概念密切相关，这正是关系数据库名称的由来

□常见的关系数据库

– Oracle 、 Microsoft SQL Server、 IBM DB2、 Microsoft Access、 MySQL、 SQLite

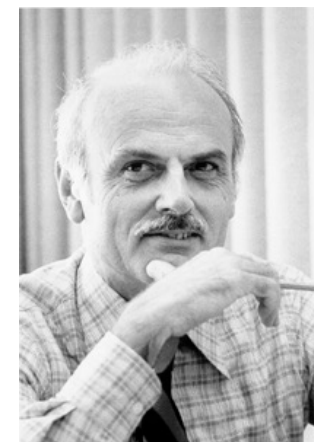
□关系数据库采用关系模型作为数据的组织方式

Student表

Sno (学号)	Sname (姓名)	Sgender (性别)	Sage (年龄)	Sdept (所在系)
2021310721	李博	男	17	CS
2021310722	赵宇	男	19	CS
2021310723	张敏	女	18	CS
2021310724	王勇	男	18	MA
2021310725	刘佳	女	17	MA

关系模型和关系数据库

- 1970年，IBM公司的埃德加·科德（Edgar F. Codd）首次提出了关系模型的概念。
- 之后，提出了关系代数和关系演算的概念
- 1972年提出了关系的第一、第二、第三范式
- 1974年提出了关系的巴斯科德范式
- 1981年，ACM授予埃德加·科德图灵奖，以表彰他在关系数据库研究方面的杰出贡献。

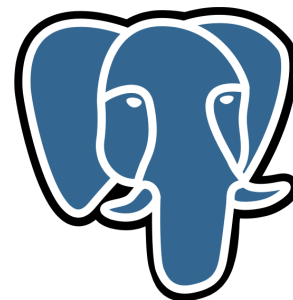


埃德加·科德

数据库发展史概览

- 层次和网状数据库
- 关系数据库
- OLTP、OLAP与HTAP数据库
- NoSQL数据库
- 分布式数据库
- 内存数据库
- 云数据库
- 其它类型数据库
 - 实时数据库
 - 并行数据库
 - 终端数据库

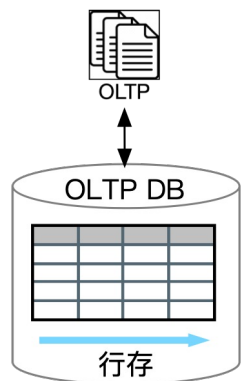
ORACLE®
DATABASE



OLTP、OLAP与HTAP数据库

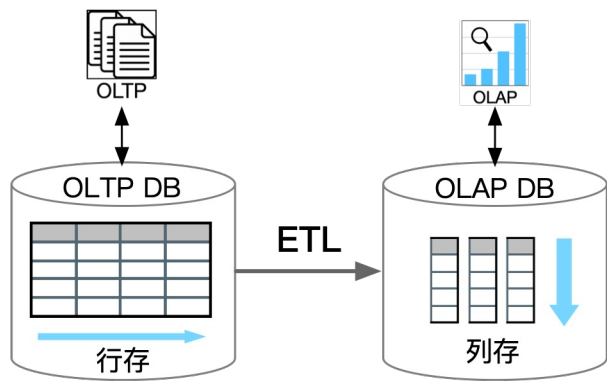
□ OLTP、OLAP与HTAP数据库发展历程（随着应用场景、数据量等特点的衍化）

阶段一



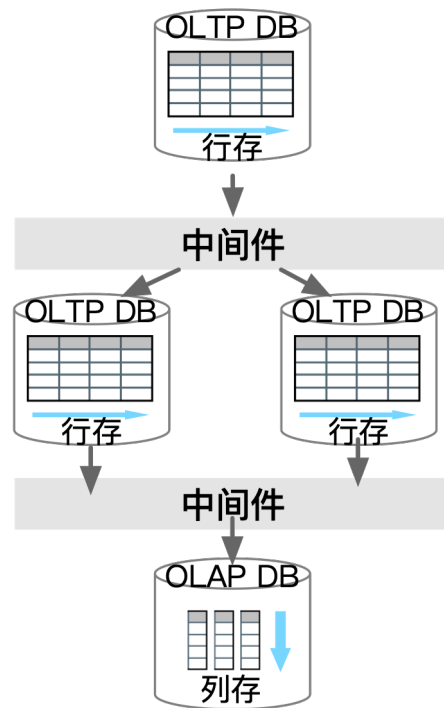
- 动机：主要为了处理业务数据
- OLTP每一条写入操作往往对应一条业务记录。

阶段二



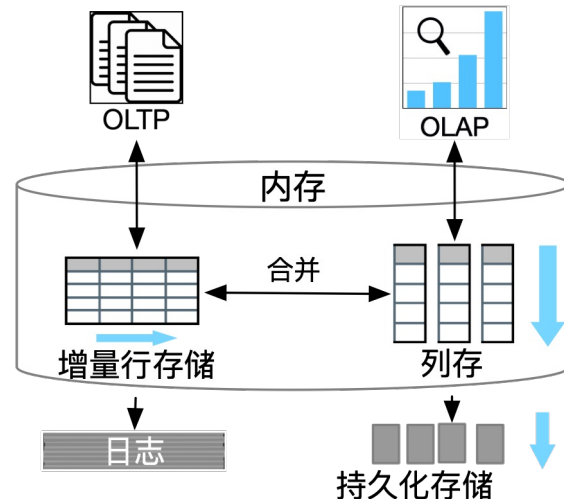
- 动机：如何从日积月累的大量数据中做数据分析，挖掘数据的价值
- OLAP定期从OLTP经过ETL提取数据，用于数据分析。

阶段三



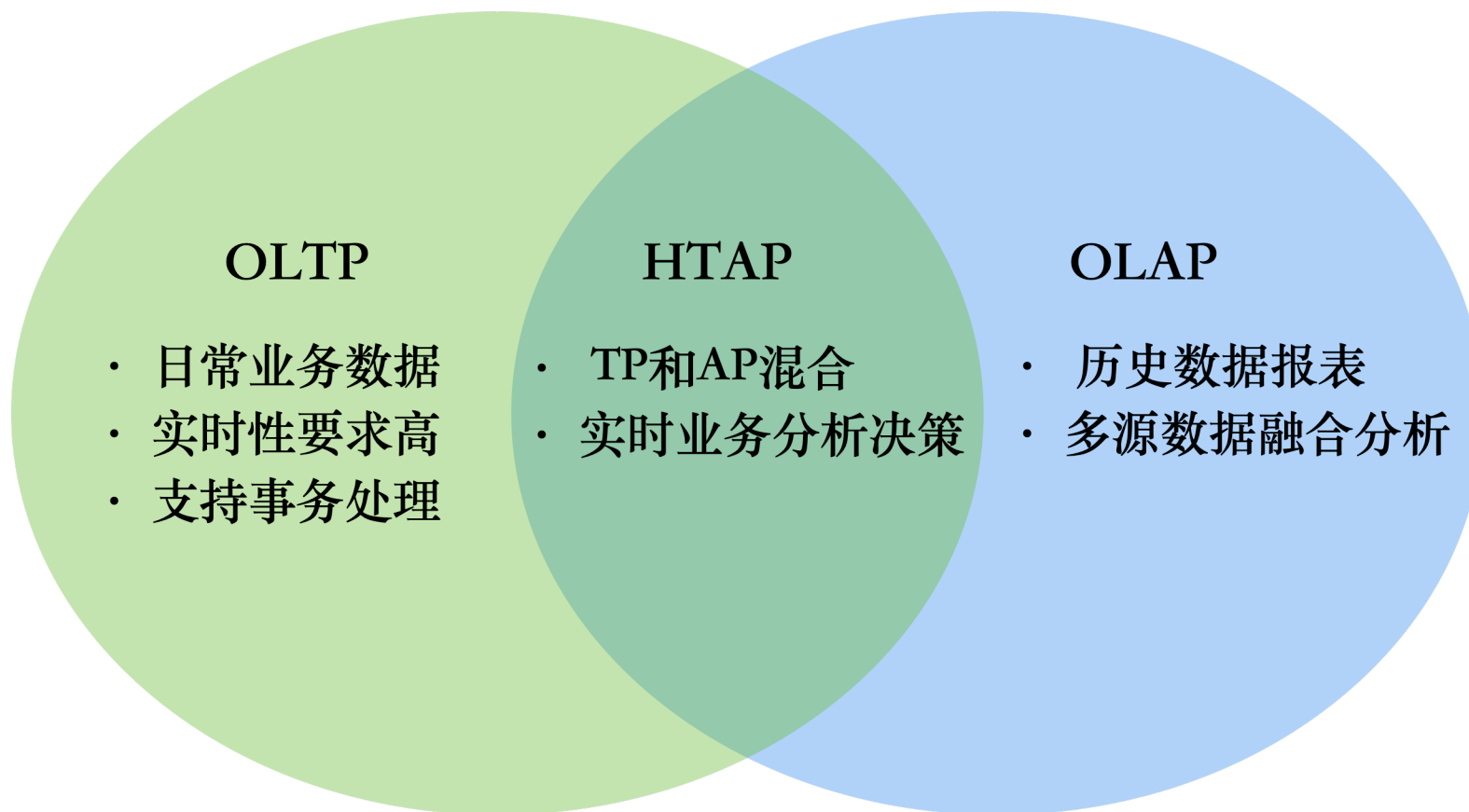
- 动机：数据业务变得复杂，单个数据库难以满足
- 数据库厂商将原有的数据库分表，通过横向扩展来满足企业的需求。

阶段四



- 动机：（1）数据量进一步增加，通过中间件进行分布式扩展极大消耗人力物力且数据同步时延大；（2）越来越多的场景需要对最新的事务信息作出快速的分析，甚至在有的情况下，需要在处理事务的同时对数据进行分析。
- HTAP是一种能够同时处理并发型事务与分析型查询的数据库技术。47

OLTP、OLAP与HTAP数据库主要特征



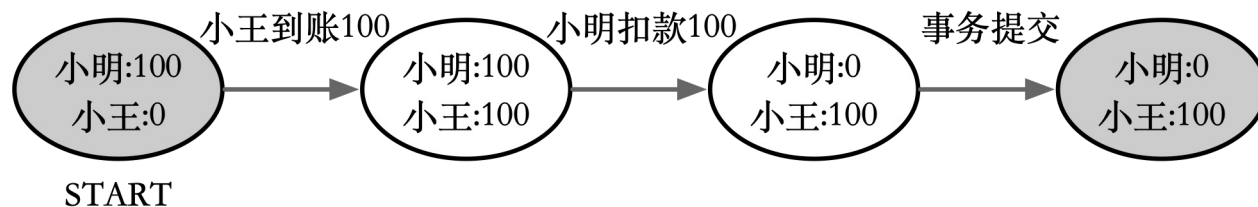
OLTP数据库

□OLTP数据库

- OLTP数据库是关系数据库的主要应用，目标是高效地进行日常的事务处理，通常每秒需要完成成千上万个查询处理（插入、更新、删除数据库中的数据），并在处理大量事务的同时需要维护数据的完整性和有效性。
- 常见的OLTP场景包括网络购物、信用卡交易处理和ATM转账等

□事务

- 事务是多个数据库操作组合成的一个不可分割的、同时成功或失败的工作单元



OLTP数据库（续）

□OLTP数据库特性

- 对实时性、并发性、安全性和数据完整性的要求高
- 单个事务处理的数据量通常不是很大
- OLTP数据库通常需要满足事务的四个特性，简称ACID特性：
 - 原子性（Atomicity）
 - 一致性（Consistency）
 - 隔离性（Isolation）
 - 持久性（Durability）

ACID特性

事务是一个不可再分的基本操作单元。事务要么全部成功要么全部失败。

原子性

一致性

事务的执行会保证数据库从一个正确（一致）的状态转移到另一个正确的状态。

多个事务在并发执行的过程中所得到的结果，和串行执行得到的结果是一致的，保证了事务并发处理中事务的正确性。并发控制是实现事务隔离性的手段。

隔离性

持久性

一旦一个事务提交以后，即使数据库发生故障，该事务的执行结果也不会丢失，可以被正确地恢复，仍然对后续事务可见

OLAP数据库

□OLAP

- **起源：**随着数据库的大规模应用，数据库内存储的数据量也日积月累地增长，于是人们逐渐开始**关心如何从这些原始数据中挖掘出隐含的价值**，因此，OLAP数据库应运而生。
- **用途：**OLAP 的常见用途包括数据挖掘和其他商业智能应用程序、复杂的分析计算和预测场景，以及财务分析、预算和预测计划等业务报告功能。
- **常见操作：**最基本的操作是对数据进行聚集和统计，这种聚集查询可能对多表进行连接并匹配和统计百万行记录。

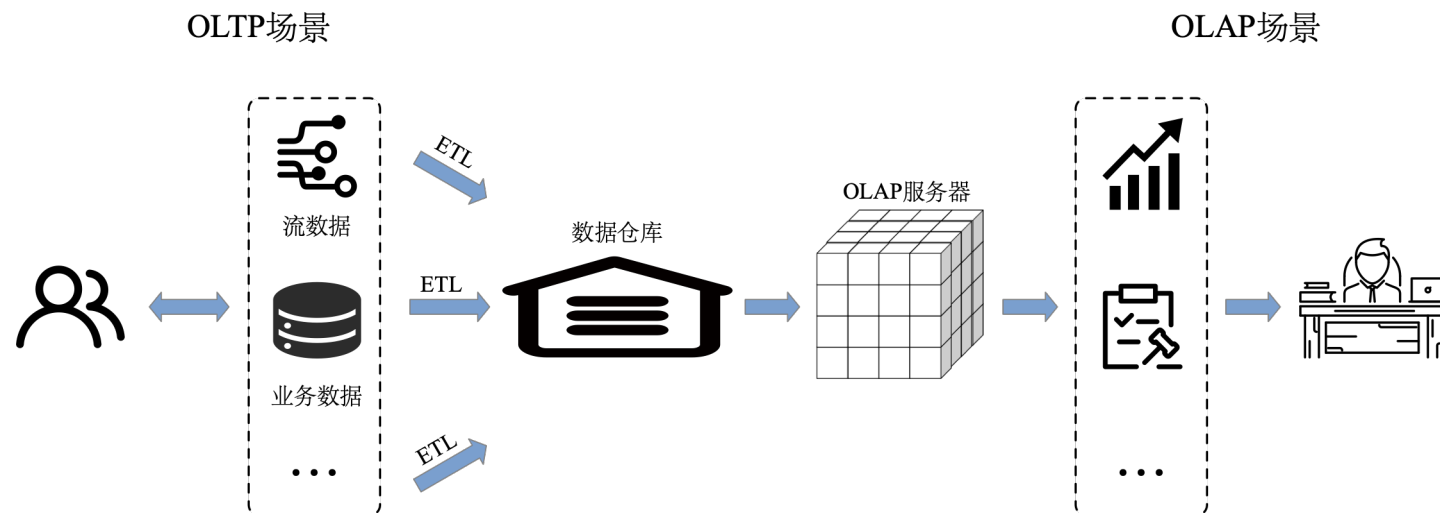
OLTP数据库 和 OLAP数据库

维度	OLTP	OLAP
主要用途	支持实时交易数据的存储、更新和共享	基于历史数据进行分析，发现知识和规律
典型场景	快速处理高并发、小批量的数据	使用复杂的查询处理大量数据
主要用户	银行柜员、收银员、仓库管理员等	数据分析师、业务分析师等
主要操作	基于INSERT、UPDATE、DELETE命令	基于SELECT命令聚合数据用以分析
数据规模	实时数据、通常较小	聚合历史数据，通常较大
并发访问	高	低
响应时间	毫秒	秒，分钟或者小时（基于场景和数据量）
存储结构	通常为行存储	通常为列存储
备份模式	需要定期备份以确保业务连续性	可以从OLTP数据库重新加载丢失的数据

OLTP—数据仓库—OLAP

□ 端到端的OLAP部署

- 在OLTP场景下，用户在不同的业务系统中产生业务数据；
- 数据仓库收集整合不同源的数据进行统一存储；
- OLAP服务器对这些数据进行统一管理并支持上层分析操作
 - MOLAP
 - ROLAP
 - HOLAP
- 最终这些分析结果将用于公司决策。



数据仓库 和 数据库

数据仓库的主要功能是从事务系统、数据库和其它数据源定期进行数据的集成、转换和管理，为上层的OLAP等数据分析任务提供支持

维度	数据仓库	数据库
主要场景	OLAP	OLTP
数据源	多数据源：异构或标准化数据	单数据源：事务系统
数据获取	通常按照预定的批处理计划 执行批量写入操作	根据工作负载动态连续 写入操作数据
数据标准化	非标准化模式	高度标准化的静态模式
数据存储	使用列式存储，优化读操作	优化写操作

数据仓库 和 数据湖

数据仓库：多源数据进行加工后集中存储的一个“仓库”。

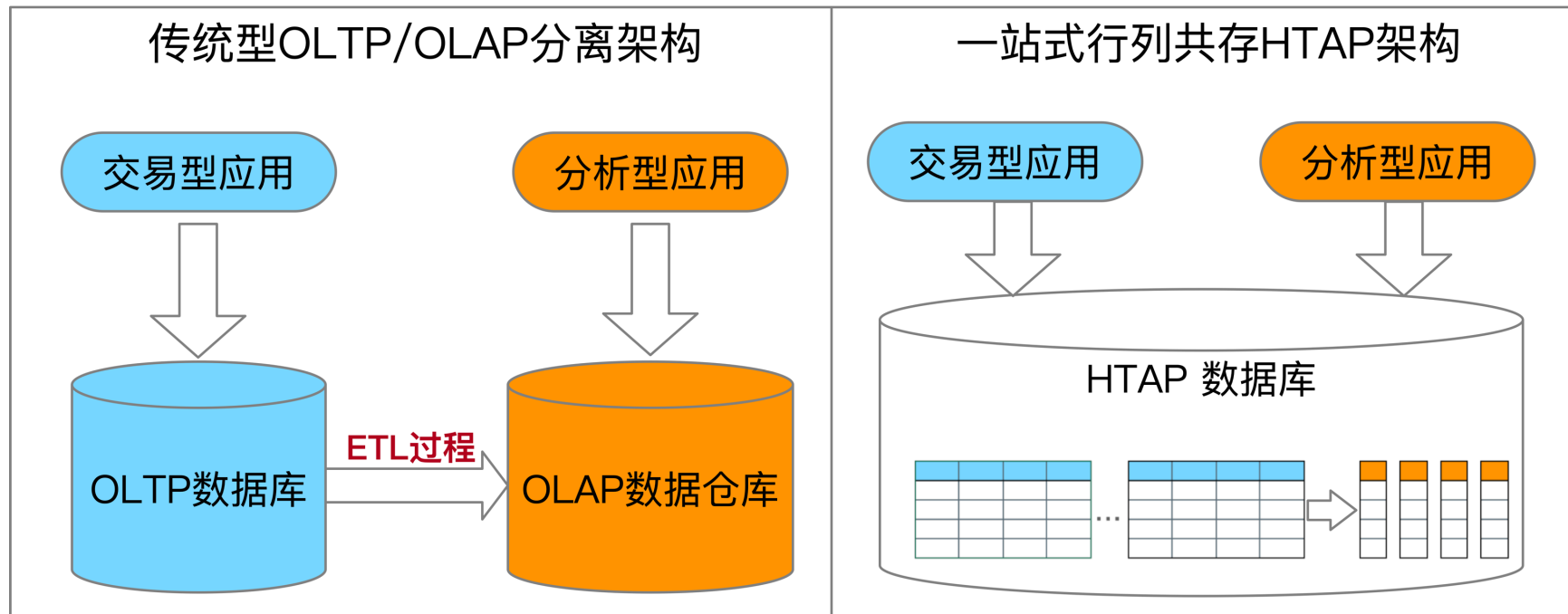
数据湖：汇集并存储多个来源的原始数据的“湖泊”，是一个集中存储和管理来自多个数据源的结构化、半结构化和非结构化数据的大型系统。

维度	数据湖	数据仓库
应用场景	数据发现、探索性分析 机器学习、大数据分析	基于结构化数据的数据分析
数据来源	所有原始数据 (含结构化、半结构化和非结构化)	来自事务系统、数据库、 业务程序的数据
数据质量	包含大量原始数据， 使用前需要清洗和标准化	数据质量高，可作为事实依据
适用对象	数据科学家、数据开发人员为主	业务分析师为主

HTAP的产生背景

□混合事务与分析处理(Hybrid Transactional and Analytical Processing)

- 著名咨询公司Gartner于2014年正式定义HTAP为: 基于内存计算技术并发地支持查询分析处理与事务处理, 并且能够实时分析最新的事务数据

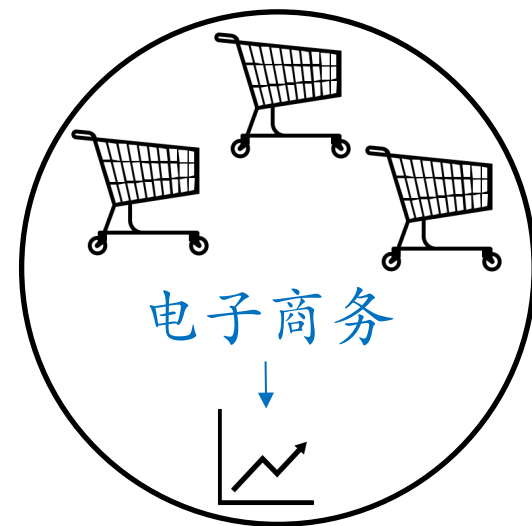


HTAP的相关应用

□ Gartner预测在2024前，HTAP技术将实时数据分析型应用领域被广泛采用

□ HTAP技术适用于需要同时保证高性能的事务处理与高度实时性的数据分析的应用

□ 例如，在电子商务应用中需要在线分析销售趋势的场景；在欺诈识别应用中需要在处理不同客户事务的同时识别出欺诈行为

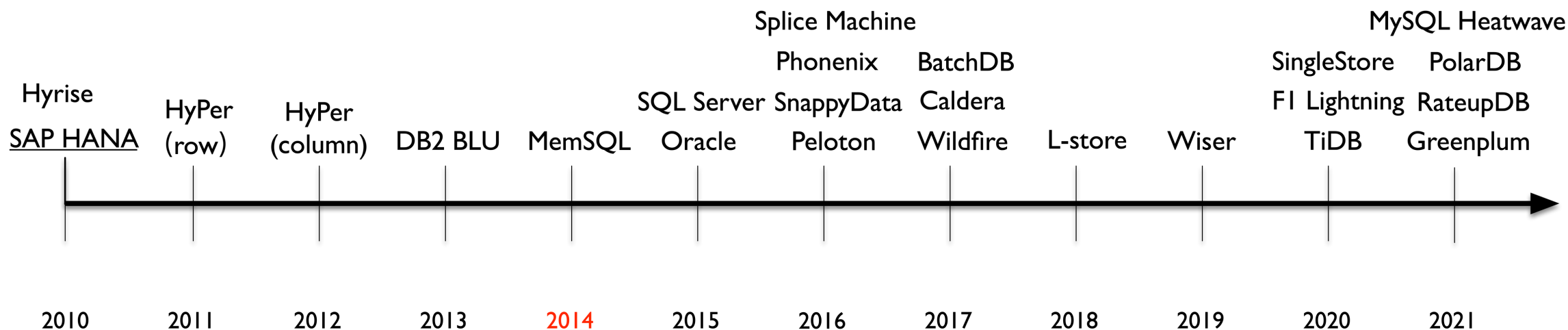


HTAP数据库的发展历程

□在过去的十多年里,已经出现了许多的HTAP数据库

□总体来说, HTAP数据库的发展历程可分为以下三个阶段:

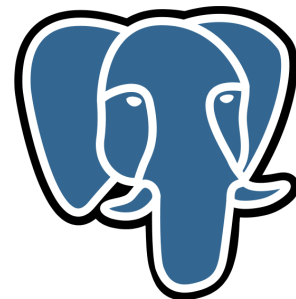
- **第一阶段** (2010-2014): HTAP数据库主要采取基于单机的主列存储
- **第二阶段** (2014-2020): HTAP数据库主要在单机行存储的基础上扩展列存储
- **第三阶段** (2020-目前): HTAP数据库主要在分布式行存储的基础上扩展列存储



数据库发展史概览

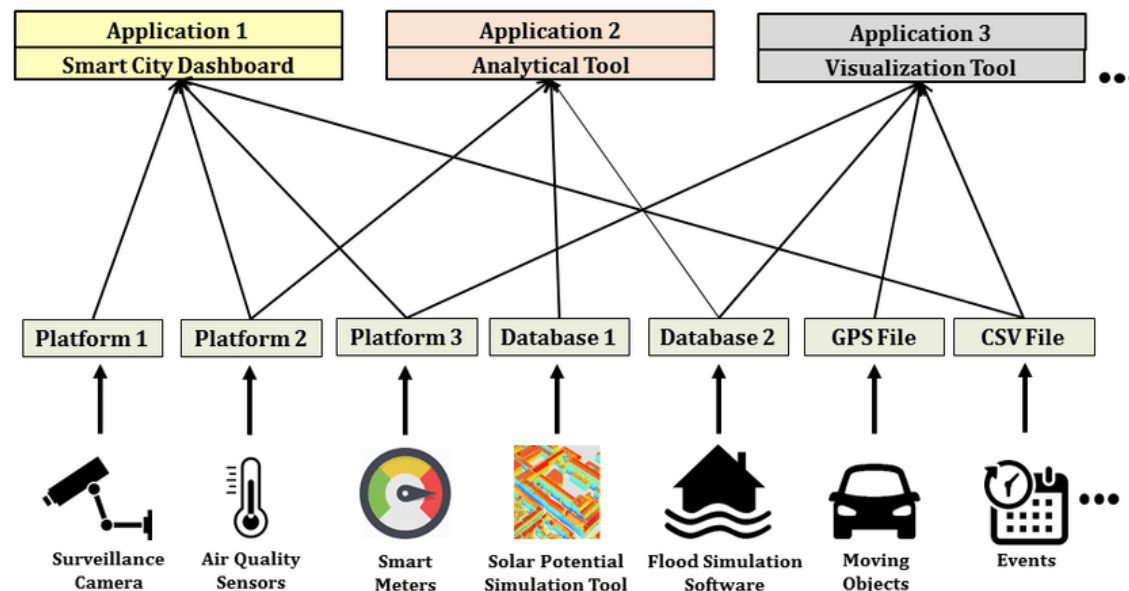
- 层次和网状数据库
- 关系数据库
- OLTP、OLAP与HTAP数据库
- NoSQL数据库
- 分布式数据库
- 内存数据库
- 云数据库
- 其它类型数据库
 - 实时数据库
 - 并行数据库
 - 终端数据库

ORACLE®
DATABASE



NoSQL数据库的产生背景

- 互联网时代以来，数据指数级爆炸性增长
- 半结构化、非结构化等异构数据处理需求增长
- 关系型数据库的强一致性并非所有场景的刚需



NoSQL数据库主要特点

□模式灵活

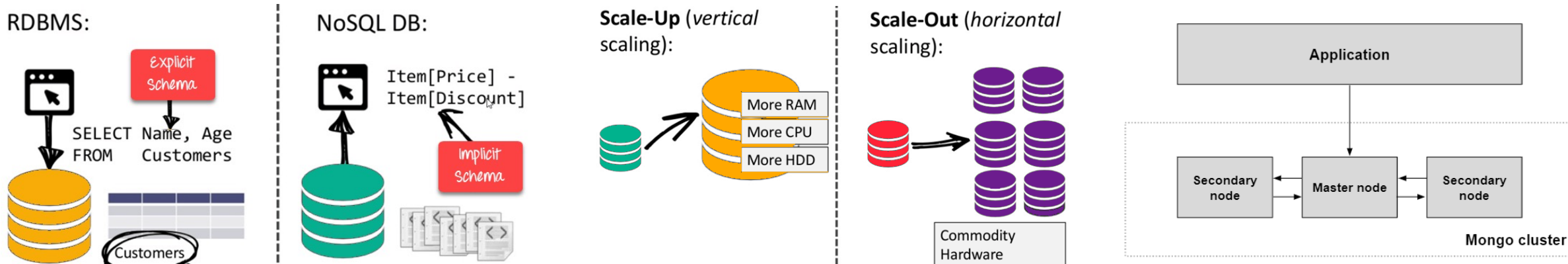
- NoSQL数据库无需预先指定数据的模式，并支持混合数据类型

□高可扩展

- 摆脱关系模型约束，NoSQL简化提升了数据水平扩展能力

□高可用

- 基于主主或主从集群模式，支持用户从不同数据节点查询数据



NoSQL数据库类型

□ 键值数据库

- 以键值对格式存储数据的数据库，数据访问简单高效

□ 列族数据库

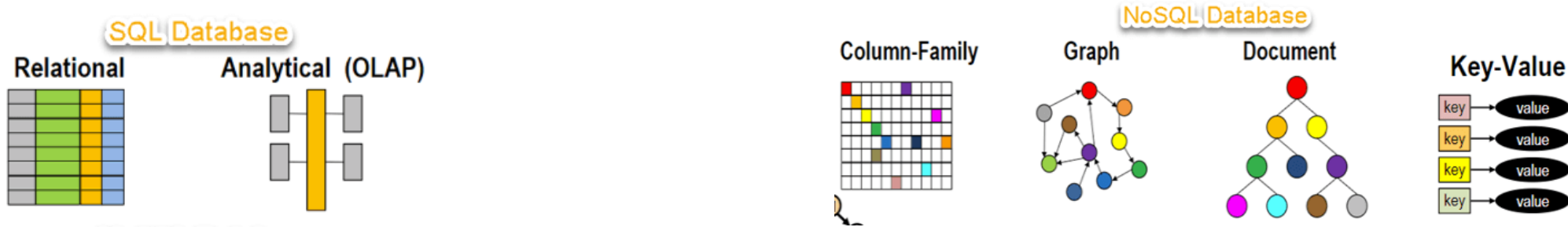
- 数据列按访问关联性划分成不同族，一个族可视为关系表

□ 文档数据库

- 以JSON等文档格式存取数据，避免应用中数据转换的代价

□ 图数据库

- 以包含边、节点及属性的图结构来存储数据和建模关联信息



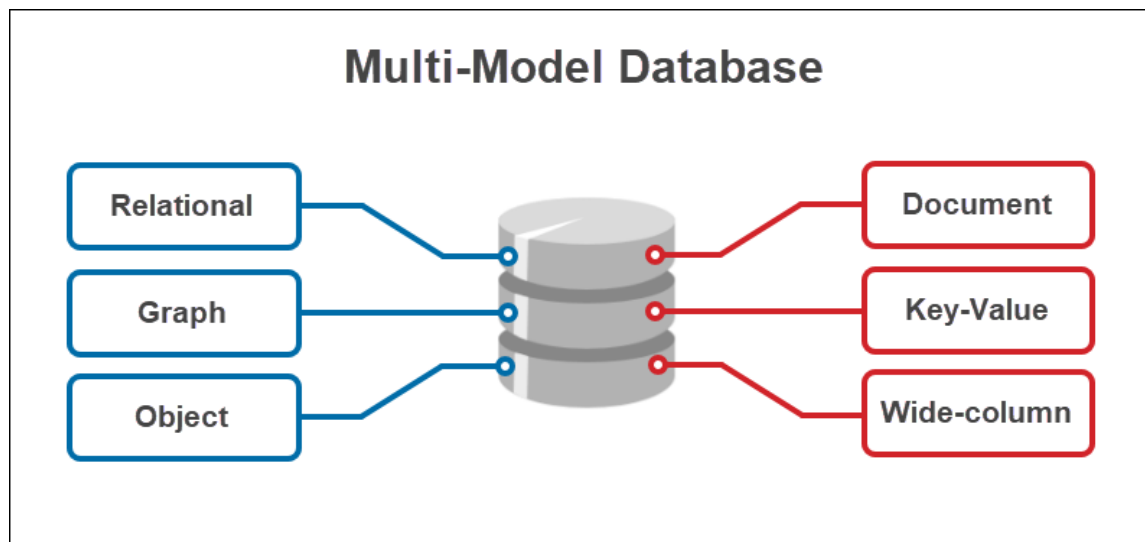
NoSQL数据库类型（续）

□时序数据库

- 优化存储和处理时序数据，设计对时序数据的压缩与查询算法

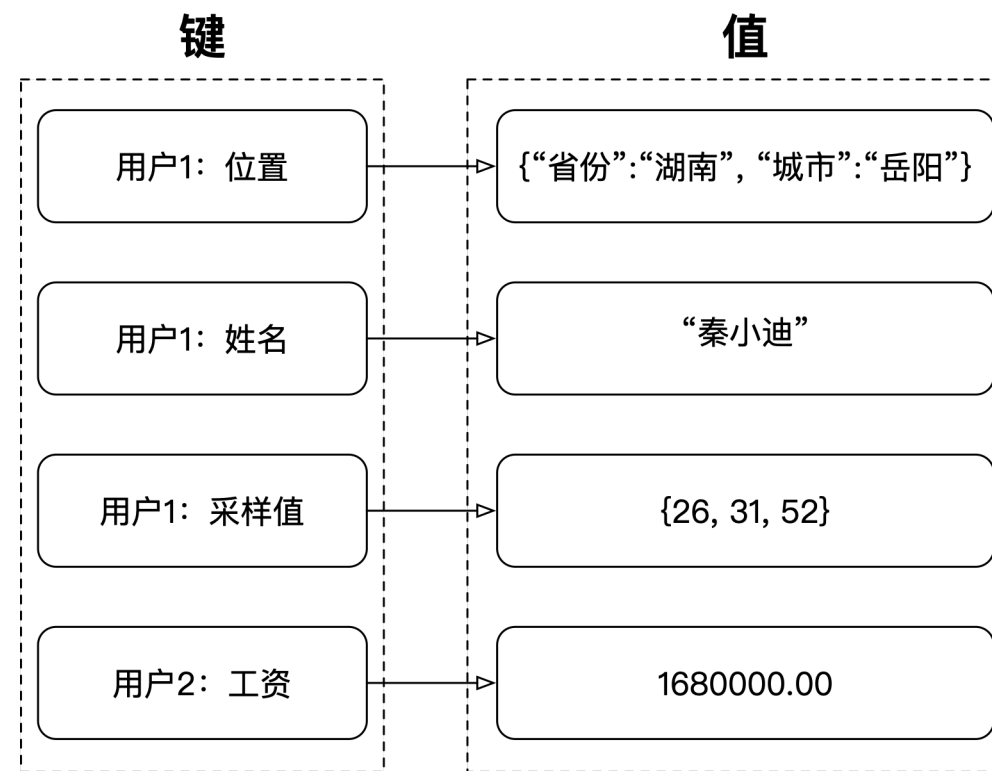
□多模数据库

- 指同时支持上述多种数据模型的数据库，以实现多种特性支持



键值数据库——定义

- ❑ 键值数据库是用键值对结构去存储数据的NoSQL数据库
- ❑ 键和值均可以是任何对象，如简单的数值和包含多个属性的复杂类的对象
- ❑ 值作为二进制大对象存储，值的语义解析需读取数据后自行解码
 - 例：键 = “用户1：姓名”
值 = “秦小迪”



键值数据库——应用场景

□ 互联网应用会话缓存

- 每个互联网应用的会话会被赋予一个唯一的会话ID，关于一个会话的信息以二进制整体存于会话ID键对应的值中，通过键索引值的会话信息读操作有极快的访问速度

□ 频繁用户/购物车缓存

- 每个用户/购物车拥有唯一的标识符，包含地址、收藏等个性信息，是高频访问信息，通过键值数据库缓存以提高速度体验

□ 不适用场景

- 数据间存在关联信息的场景不适合使用键值数据库，比如多个键关联查询的场景，这些场景需要完成数据或键之间的关联操作

列族数据库——定义

□列族数据库按查询需求组织数据列，由一到多个列族组成，每个列族可视为一个关系表

□列族数据库避免了分析等查询加载不相关列数据，有效提升性能

- 例：列族数据库中，学生的姓名列和成绩列是分属于两个不同列族的，因此存储不相邻。

关系数据库：按行存储

学号	姓名	性别	GPA
2021310721	李博	男	3.9
2021310722	赵宇	男	3.7
2021310723	秦迪	女	4.0

列族数据库：按列族存储

列族1

学号	姓名	性别
2021310721	李博	男
2021310722	赵宇	男
2021310723	秦迪	女

列族2

学号	GPA
2021310721	3.9
2021310722	3.7
2021310723	4.0

列族数据库——应用场景

□ 互联网应用事件日志

- 列族数据库常被用来存储应用的事件日志信息，它能够支持存储任何数据结构形式的时间日志信息，比如企业所有应用的状态信息、错误日志等存储到列族数据库中，实现写高可扩展性

□ 博客平台数据存储

- 列族数据库适合博客等内容管理与存储，通过访问需求来划分列族存储用户信息（账号、姓名、年龄等）、博客信息（标签、类别、链接和引用等）和评论信息（文本、点赞、回复等）。

□ 不适用场景

- 不适合读写操作需要满足ACID特性的应用场景，不适用对业务频繁查询、列关联访问情况了解不充分的情况

文档数据库——定义

□文档数据库是将数据作为类JSON数据存储和查询的数据库，简而言之即层次化的键值对结构

□类JSON格式优点

- 数据存储与应用程序中对象结构一致，大大减轻了应用开发过程中数据建设的工作

□与键值数据库的关系

- 文档数据库可视为键值数据库的扩展，使其可部分获取层级嵌套子对象，随应用需求灵活变化存取的内容

文档 1
{ “学号”: “2018310721” “姓名”: “李博” “性别”: “男” “GPA”: 3.9 }

文档数据库——对比关系数据库

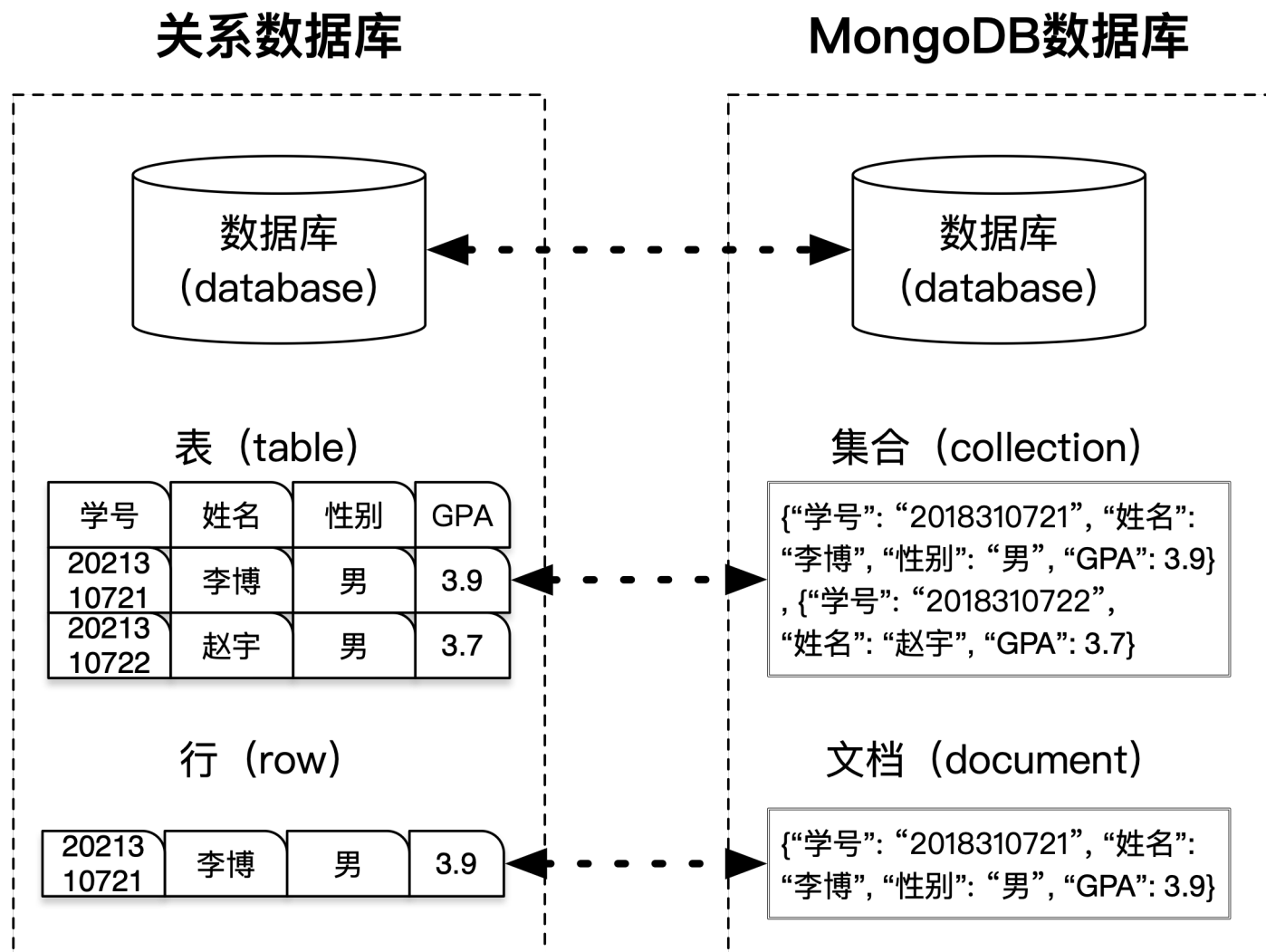
- 一个文档数据库实例可包含多个文档数据库
- 一个文档数据库可包含多个数据集合
- 一个数据集合可包含多个文档



Couchbase



amazon
DynamoDB



文档数据库——应用场景

□用户个人信息存储维护

- 文档数据库有灵活可变的数据模式，它适合不同用户提供不同类型信息的在线用户画像场景，而无需存储空值字段

□异构业务内容管理

- 由于文档数据库的模式灵活性，它非常适合异构业务发展，用来融入新的内容类型，比如图片、评论和视频等用户生产的内容

□不适用场景

- 不适合需要较多涉及多个文档的复杂事务查询的业务
- 不适合属性列一直变化情况，因为查询字段也需要不断变化

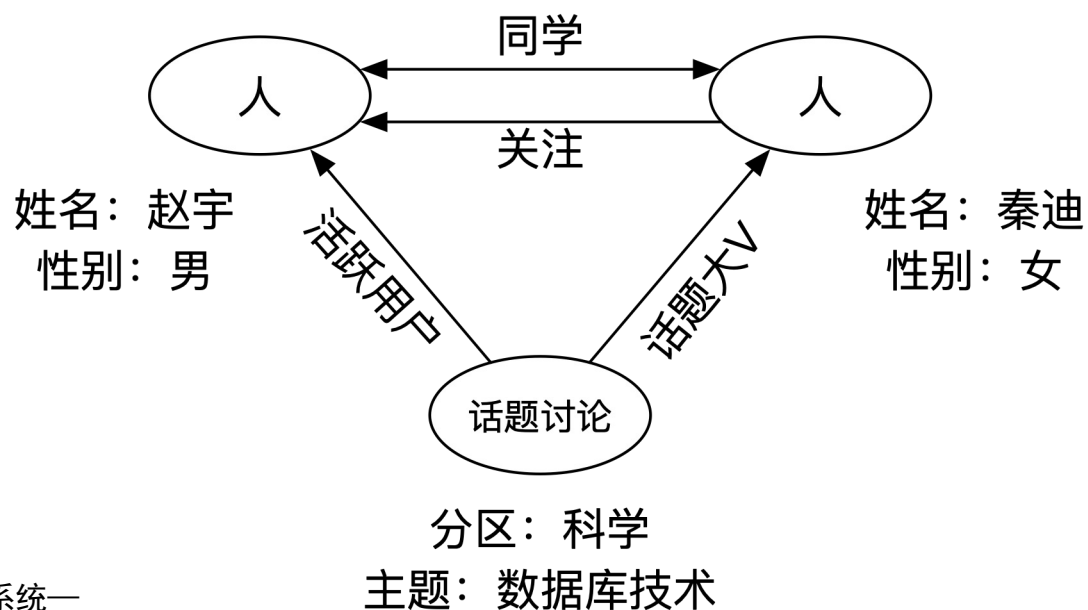
图数据库——定义

□图数据库存储实体之间的关系，其中实体是图的节点，实体之间的关系是边，节点和边拥有属性信息

- 节点代表的应用的对象实例，边代表的是应用对象之间的关联，关联有着方向性

□用途：图数据库主要用于基于关联信息挖掘有价值的规律

- 节点：“人”和“话题讨论”
- 边：“同学”和“活跃用户”
- 属性：“姓名”、“主题”



图数据库——应用场景

□ 社交网络

- 图数据库是有效部署社交网络数据的方式，社交网络数据不只是包含朋友关系，还包含雇佣关系，亲属关系等等关联

□ 时空信息

- 对于空间路由和快递场景，图数据库可以用于查找和优化路径，存储历史结果。对于基于位置的服务，图数据库可以用于查询推荐用户感兴趣的内容，比如附近的餐馆、娱乐设施等

□ 不适用场景

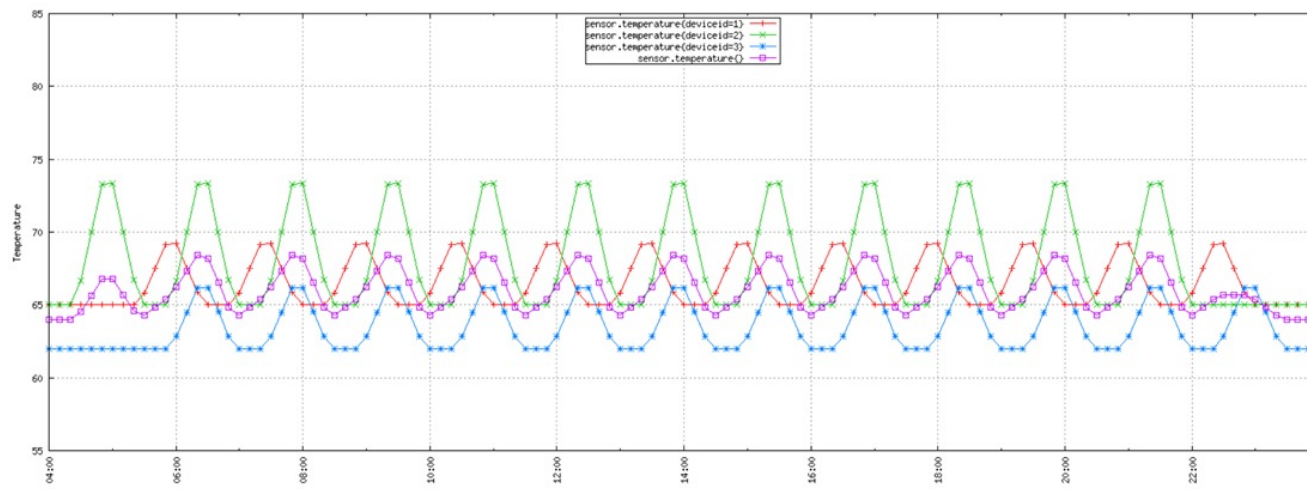
- 图数据库面向关联查询，未针对数据存取优化，不适合用于业务数据的场景，这其中会有大量的事务查询操作

时序数据库——定义

□时序数据库是针对时序数据的存储和查询进行优化的数据库，其中时序数据指的是伴随有时间戳的数据

□时序数据库特性

- 基于时序数据均匀分布的特点去设计专用高效的数据压缩算法，实现比通用压缩算法对非均匀数据更高的性能
- 基于时序数据均以时间戳索引的特点，可实现更高查询性能



时序数据库——应用场景

□物联网传感器数据

- 比如水、能源和温度数据的采集和异常值监控。时序数据库可以支持高效的时序数据分析，识别季节性、均值等数据统计特性

□监控互联网服务

- 监控互联网服务、应用及部署的基础设施的性能和服务状态等信息。比如监测分布式业务的异常流量，节点故障等关键信息，以实现异常问题的快速响应

□不适用场景

- 不具备时序特性或者时序信息非数据关键特征的数据场景不适合使用时序数据库

多模数据库——定义

- 多模数据库是集成同时支持多种数据模型的数据库系统
 - 支持多种数据类型具体包含文档、图、关系数据和键值数据等
- 多模构建方式
 - 深度集成：在同一个数据查询引擎上支持多个数据模型
 - 中间件集成：支持的不同数据模型的查询引擎基础上建设通用的查询支持接口，不同数据模型的查询服务由相应的引擎支持

多模数据库——应用场景

□统一管理多种异构数据

- 多模数据库实现在单个数据库中存储多种数据模型的异构数据。比如，需要同时支持有关系数据、JSON数据和图数据搜索的应用场景

□实现多种NoSQL数据库特性支持

- 比如给JSON数据提供SQL查询接口支持，再比如对XML数据通过JSON格式访问等类似需求

□不适用场景

- 单一简单数据源应用场景，使用多模数据库只会增加复杂度，而使用专用数据模型数据库有更多特性支持，有更好性能优化

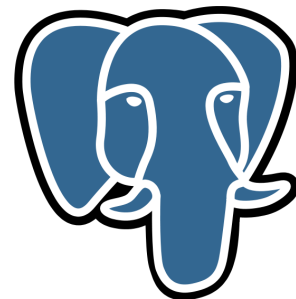
常见的NoSQL数据库

主要类型		NoSQL数据库产品（示例）
键值数据库	内存键值存储	Redis, Memcached
	最终一致性键值存储	Dynamo, Riak KV
	持久化键值存储	LevelDB, Tarantool
列族数据库		Cassandra, Hbase, Google BigTable
文档数据库		MongoDB, CouchDB
图数据库		Neo4J, FlockDB

数据库发展史概览

- 层次和网状数据库
- 关系数据库
- OLTP、OLAP与HTAP数据库
- NoSQL数据库
- 分布式数据库
- 内存数据库
- 云数据库
- 其它类型数据库
 - 实时数据库
 - 并行数据库
 - 终端数据库

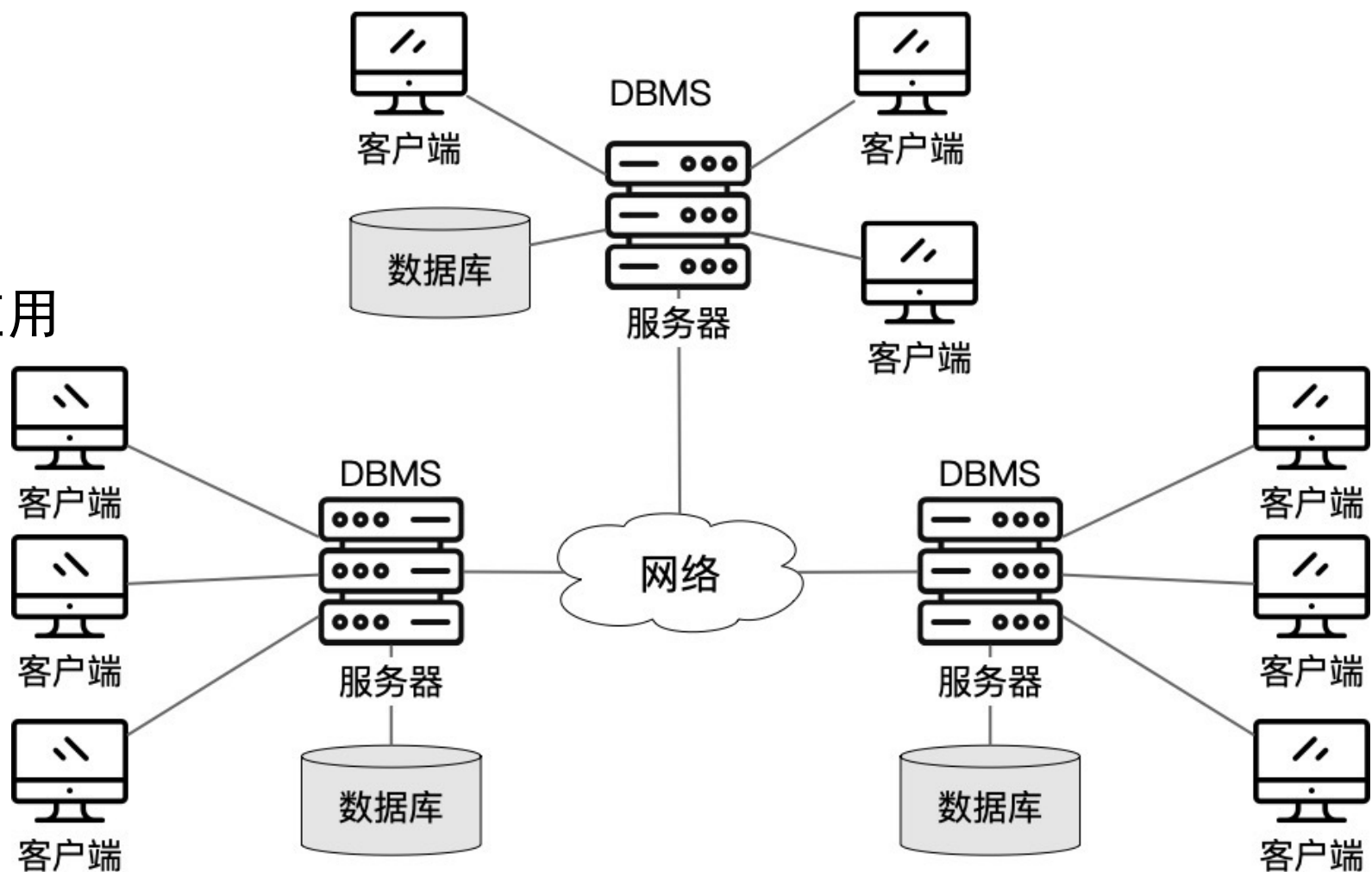
ORACLE®
DATABASE



分布式数据库

产生背景

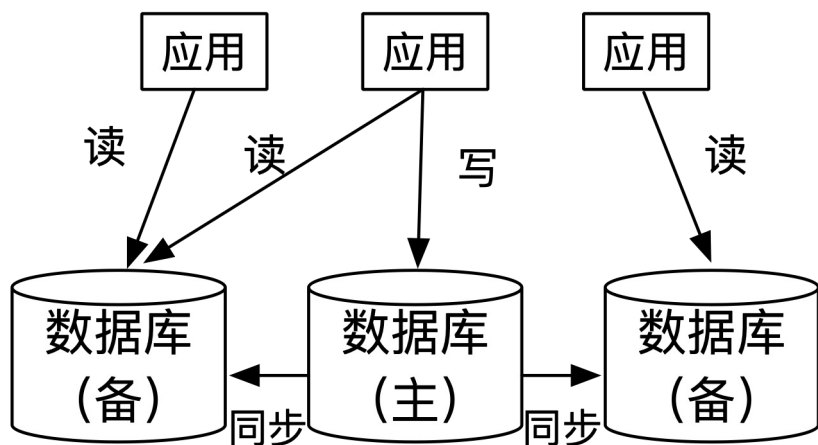
- 海量数据存储
 - TB、PB级数据
- 高可用应用需求
 - 电商、通信、金融应用
- 分布式数据库
 - 高性能
 - 高可用
 - 高可扩展



分布式数据库—常见架构

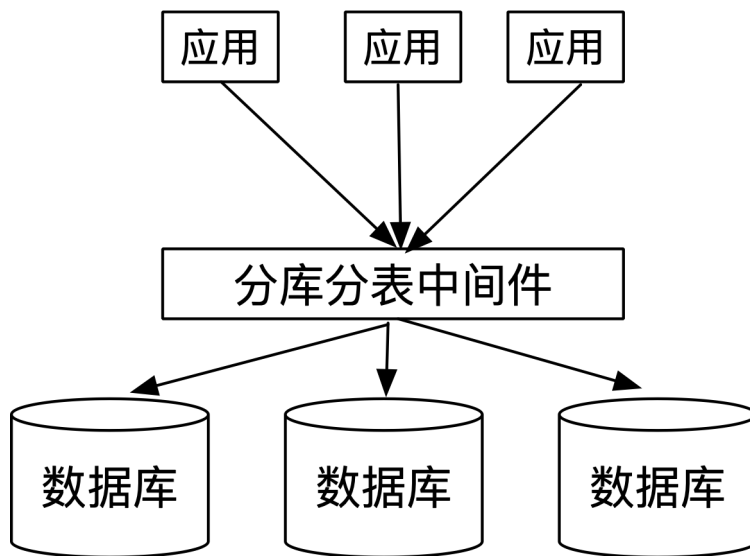
□一主多备，读写分离

- 主节点提供读写服务
- 备节点只提供读取功能，不提供写入服务



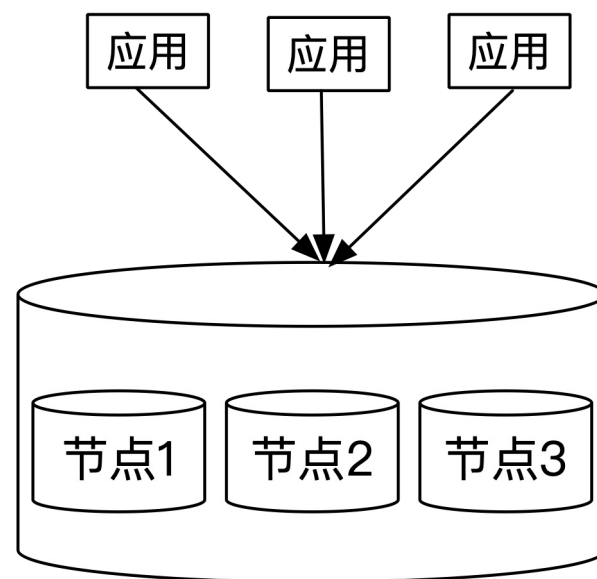
□分库分表中间件

- 搭建于多台单机数据库上层
- 负责SQL的转发



□原生分布式数据库

- 重新设计数据库架构
- 对上层应用透明

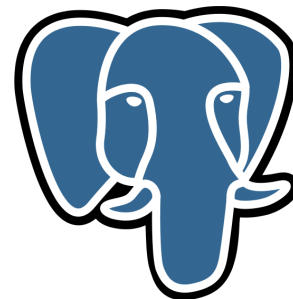


原生分布式数据库

数据库发展史概览

- 层次和网状数据库
- 关系数据库
- OLTP、OLAP与HTAP数据库
- NoSQL数据库
- 分布式数据库
- 内存数据库
- 云数据库
- 其它类型数据库
 - 实时数据库
 - 并行数据库
 - 终端数据库

ORACLE®
DATABASE



内存数据库

□起源

- 磁盘数据库I/O效率低。
 - 传统数据库系统通常是基于价格便宜磁盘进行设计，大量数据存储在磁盘中。这类数据库在进行事务处理等操作时，往往需要涉及磁盘I/O操作，很难满足实时应用系统的性能需求。
- 近年来，内存容量变得越来越大并且价格也越来越便宜。

□特点

- 内存数据库是一种在内存中进行数据存储以及数据访问控制的数据库管理系统。
- 内存数据库将数据存储在内存中，使用磁盘进行数据恢复，受益于内存在性能以及访问速度上的巨大优势，内存数据库具有高吞吐率、低延迟以及高并发的特点。

内存数据库（续）

□发展历程

1984年到1994年

- 内存数据库发展的萌芽期
- 学术界开始构思如何基于内存进行数据管理
- 受限于当时的计算机硬件水平，内存数据库并没有得到大规模的应用。

2005年至今

- 这十年陆陆续续出现了一些商用的内存数据库。早期的内存数据库主要是为了满足金融以及电信领域对数据库系统的性能需求。
- 美国贝尔实验室研制了DataBlitz内存数据库
- 1996年，Oracle发布了TimesTen内存数据库。

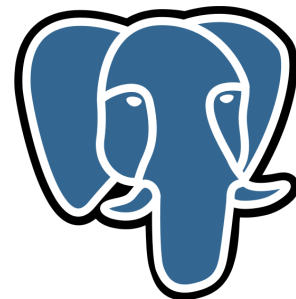
1994年到2005年

- 随着计算机硬件的进步，在内存容量增加且价格便宜，奠定了硬件基础
- 随着互联网的发展，涌现出大量的应用场景需要高性能内存数据库的支持。
- 研发出了Redis、SAP HANA、VoltDB、Hekaton和Hyper等内存数据库，这些数据库在商业市场中得到了广泛的应用。

数据库发展史概览

- 层次和网状数据库
- 关系数据库
- OLTP、OLAP与HTAP数据库
- NoSQL数据库
- 分布式数据库
- 内存数据库
- 云数据库
- 其它类型数据库
 - 实时数据库
 - 并行数据库
 - 终端数据库

ORACLE®
DATABASE



云数据库

□核心：数据库即服务

□2个角度

- 提供商：规模效应
- 使用者：3大优势

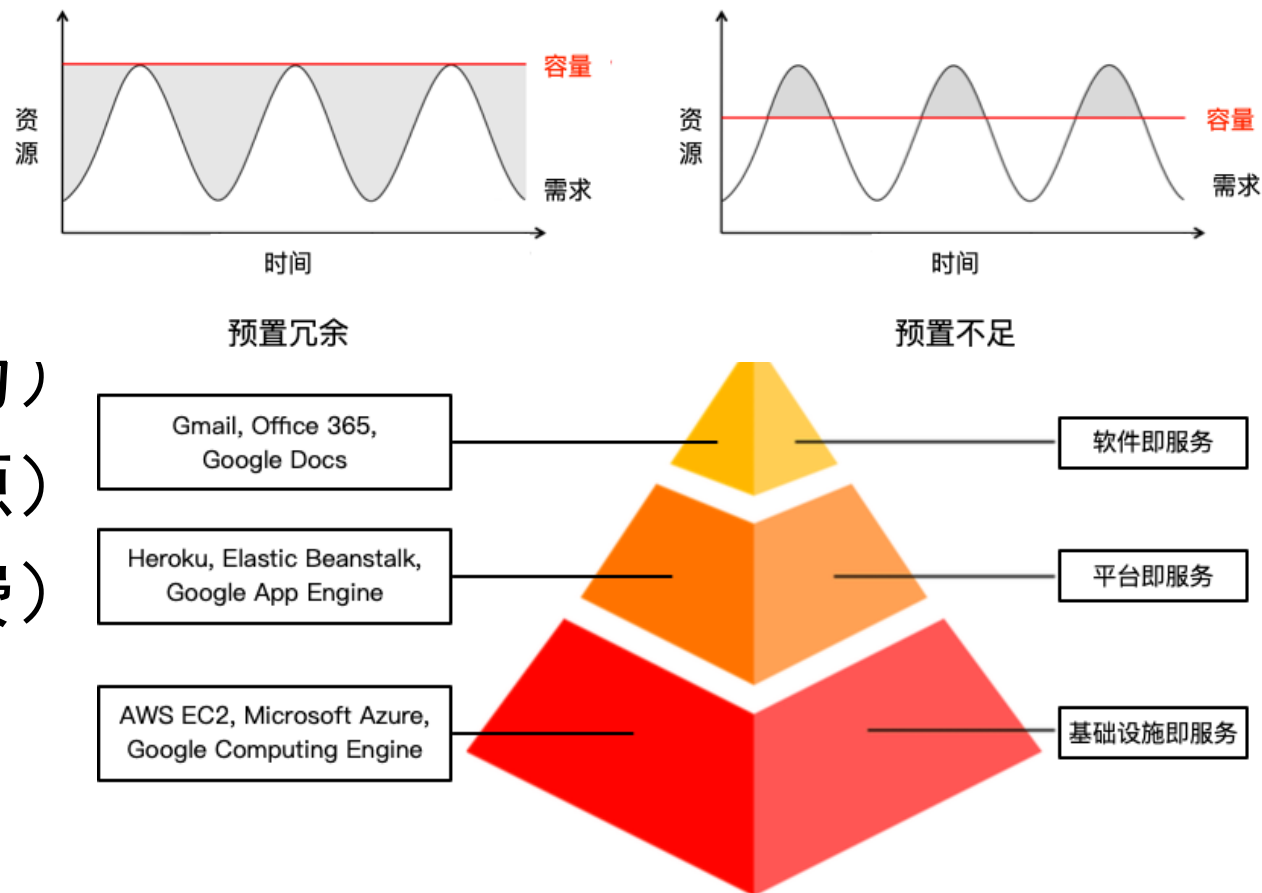
□3大优势

- 高可用，强容灾
- 弹性扩容，按需计费
- 智能运维，自动优化



云服务简介

- 云服务的背景：动态工作负载
- 云服务的层级：IaaS, PaaS, SaaS
- 云服务的特性：
 - 兼容异构的基础资源（虚拟化）
 - 支持异构的业务体系（容器化）
 - 支持动态的资源扩展（容灾能力）
 - 超大规模的业务支持（无限资源）
 - 弹性的按需计费机制（弹性计费）



云托管数据库

□云托管数据库系统特点：

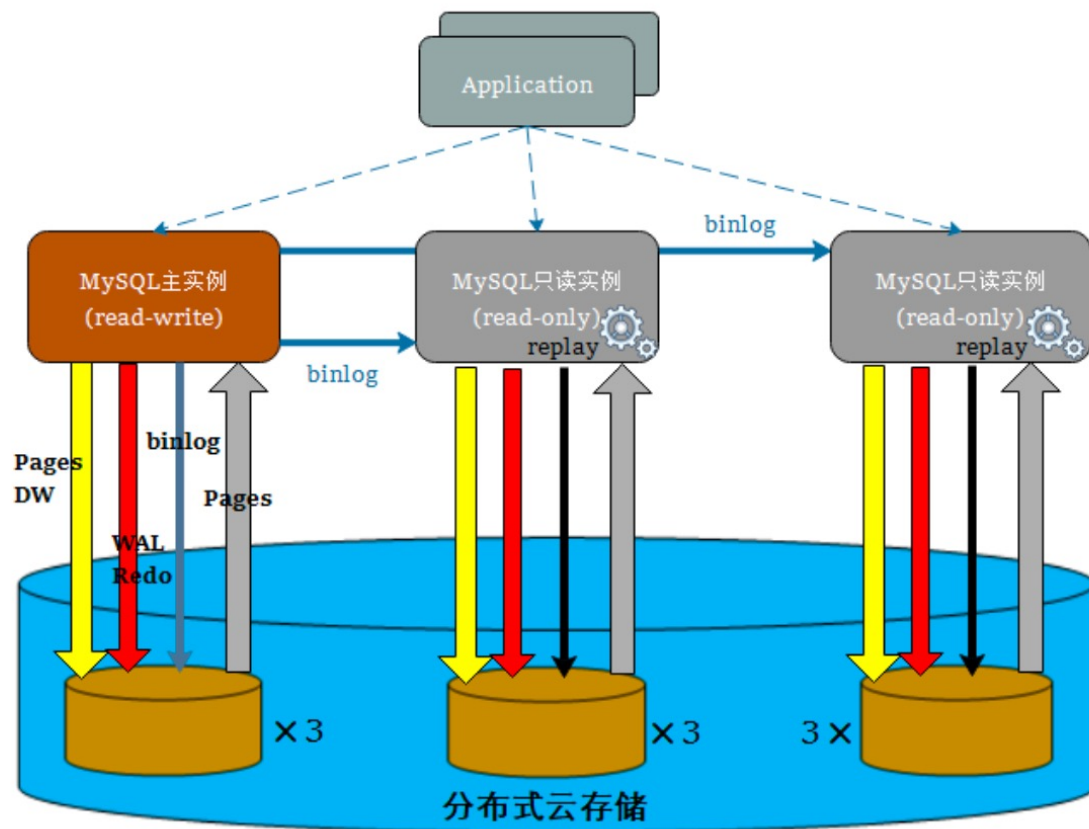
- 多个数据库实例
- 总体为Shared Everything结构

□优势：

- 易于实现，完全兼容

□缺陷：

- 本机存储容量
- 冗余的数据副本
- 高RTO，数据滞后
- 网络瓶颈



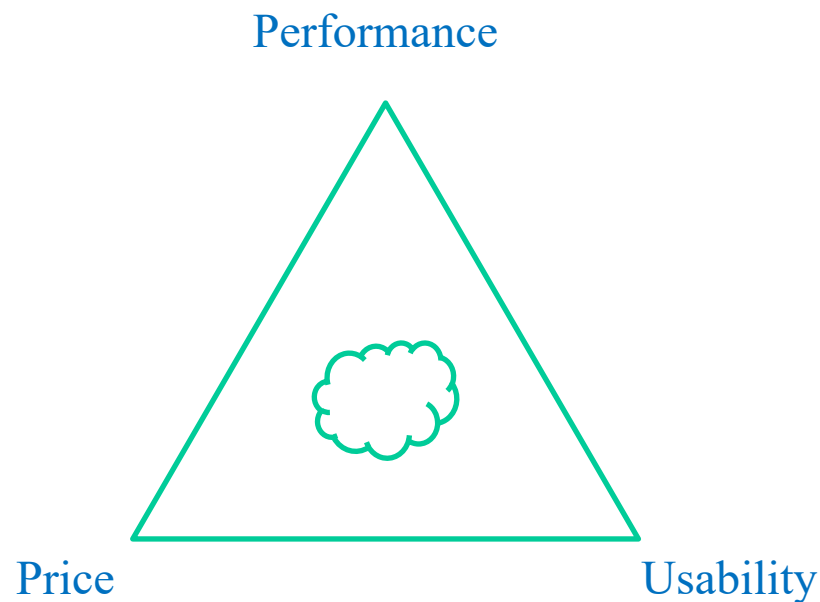
云原生数据库

□核心区别：架构设计上的计算存储分离

- OLTP:注重事务处理，日志为核心
- OLAP:强调数据分析，分片为核心

□关键技术：

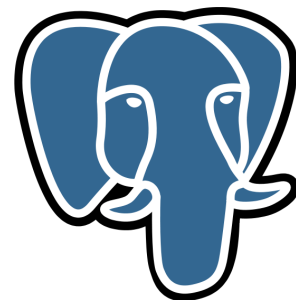
- 事务处理
- 数据副本
- 异步更新
- 故障恢复
- 分析处理



数据库发展史概览

- 层次和网状数据库
- 关系数据库
- OLTP、OLAP与HTAP数据库
- NoSQL数据库
- 分布式数据库
- 内存数据库
- 云数据库
- 其它类型数据库
 - 实时数据库
 - 并行数据库
 - 终端数据库

ORACLE®
DATABASE



实时数据库

□动机

- 在现实生活中，有许多场景要求数据库系统能够实时处理不断变化的数据信息。
- 例如，股市中每个股票的价格变动是十分快速的，这就需要数据库实时处理不同时间戳的股票交易价格。

□定义

- 实时数据库是实时系统和数据库技术相结合的数据库系统，主要面向处理快速变化的数据及具有时间限制的事务处理

□常见的实时数据库

- Apache Storm、PipelineDB、Heron和Google的Dataflow 等

实时数据库（续）

□主要特点

- 如何正确有效地表示时间？
 - 将现实世界的时间概念与实时系统的时间概念相关联
- 如何有效表示数据库中的属性值？
 - 使得数据一致性和流程事务不冲突
- 实时数据库的事务处理既要考虑事务的执行时间，也要考虑事务的截止时间和紧迫程度

并行数据库

□动机

- 提高数据库的性能和可用性

□历史

- 对并行数据库的研究可以追溯到20世纪70年代中后期的数据库机（database machine），数据库机可以存储和分析数据
- 随着数据量的增加，单机已经很难处理更大的数据集，因此研究者们想通过并行计算技术来提高单机的处理能力。

□定义

- 研究者提出利用并行处理的方法，通过多个处理节点以并行执行数据库任务，以提高整个数据库系统的性能和可用性，这类基于并行计算技术来提高数据库性能的数据库是并行数据库。

并行数据库（续）

□常见的并行数据库

- Teradata、VMware Tanzu Greenplum等
- Teradata
 - 1982年发布，是第一个商用的并行数据库系统
 - 当时的存储容量可以达到1TB
- VMware Tanzu Greenplum
 - 基于大规模并行处理（massively parallel processing, MPP）架构

□现有挑战

- 尽管并行数据库通过并行连接多个计算资源（如 CPU 和磁盘），可以显著提高数据库系统的性能、稳定性和高可用性
- 依然存在并行数据库造价高昂、系统弹性较差和管理成本较大等难点

终端数据库

□动机

- 为了解决终端设备小规模数据管理的需求，终端数据库应运而生

□特点

- 结构简单，性能高，可以作为嵌入式数据库内建在应用程序或者终端设备中
- 降低了传统数据库的“客户端-服务器”模式下高昂的数据通信开销。

□常见终端数据库

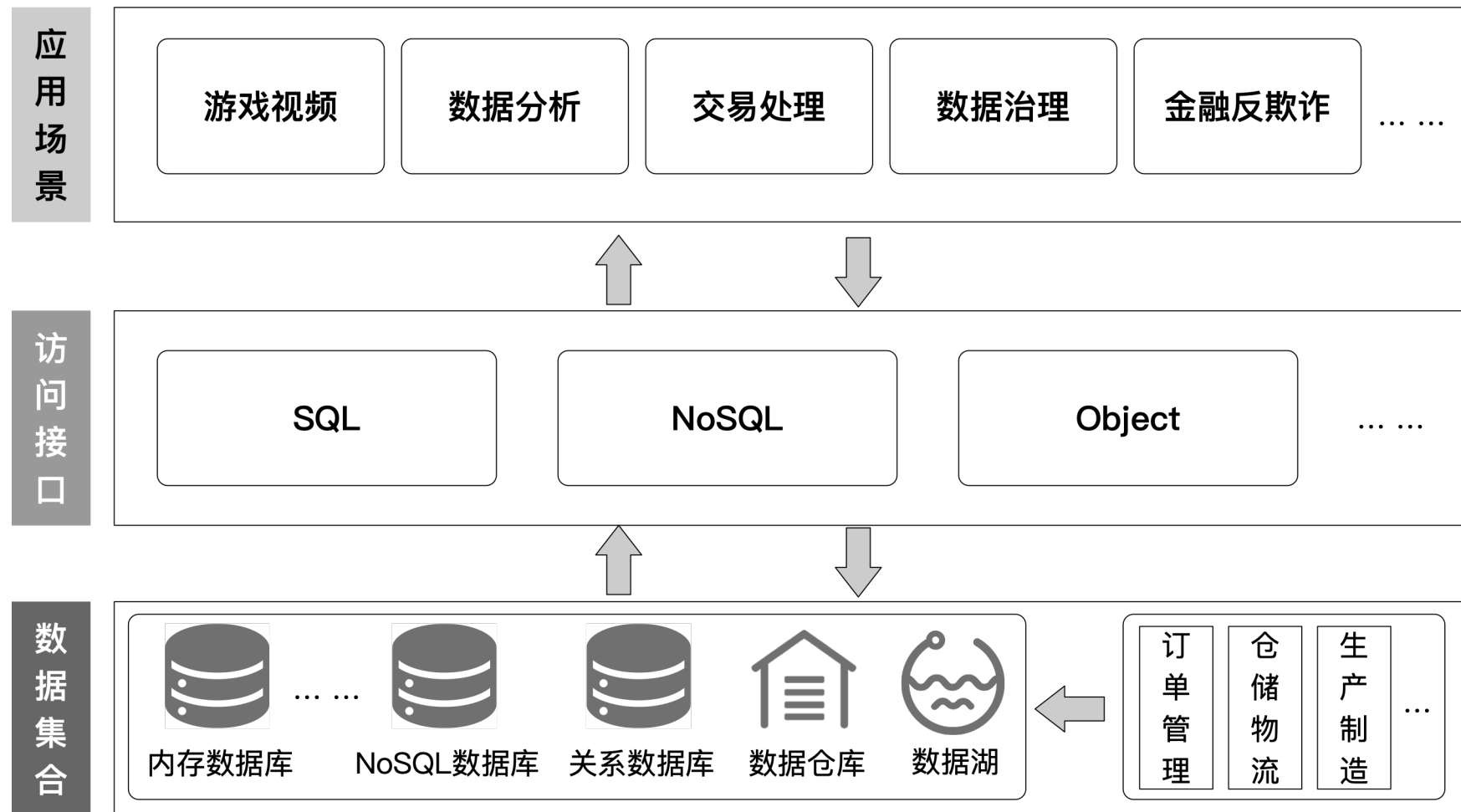
- SQLite

目录

- 数据库系统概述
- 数据库发展简史
- 数据库系统的应用和展望
- 本章小结

数据库系统的应用

- 管理信息系统
- 事务处理系统
- 决策支持系统



数据库新发展机遇

- 人工智能与数据库
- 新硬件与数据库
- 区块链与数据库

人工智能遇见数据库，能擦出哪些火花

□ 传统数据库上云，面临诸多挑战



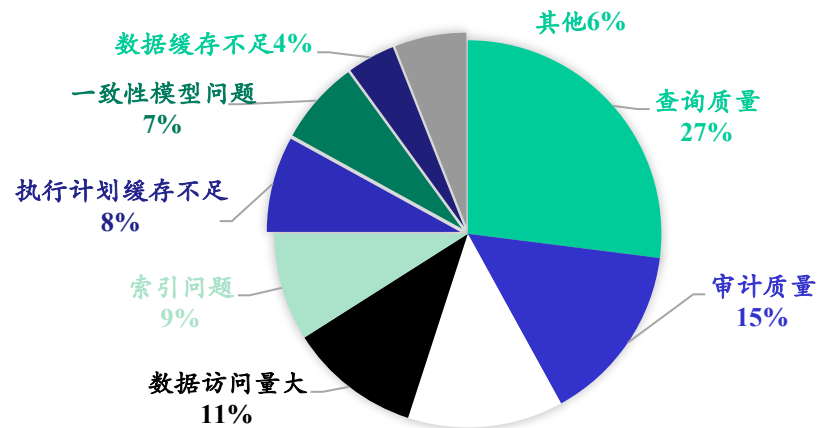
"one-size-not-fits-all"

需要根据应用自适应
调整数据库配置



"challenges in big data"

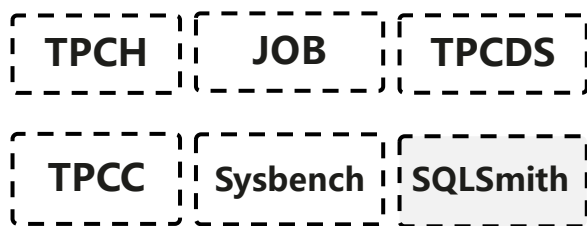
数据难划分，查询难优化
实时数仓（速度；一致性）



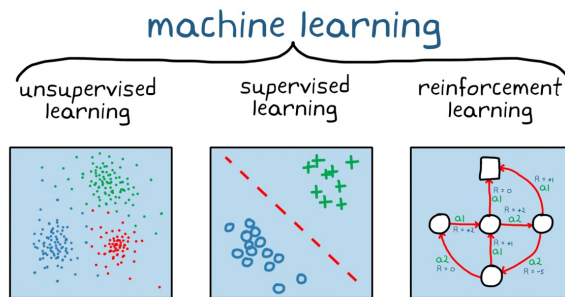
"issues in db maintenance"

人力或简单的运维工具
难以支撑云上庞杂的运维任务

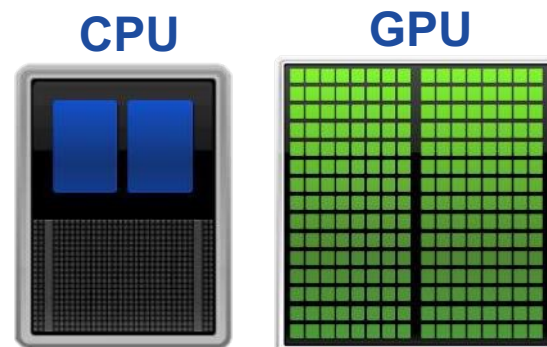
□ AI在数据、算法、新硬件的驱动下取得突破进展



训练数据管理，提效率



多种机器学习算法，库原生
数据库系统—人工智能与数据库

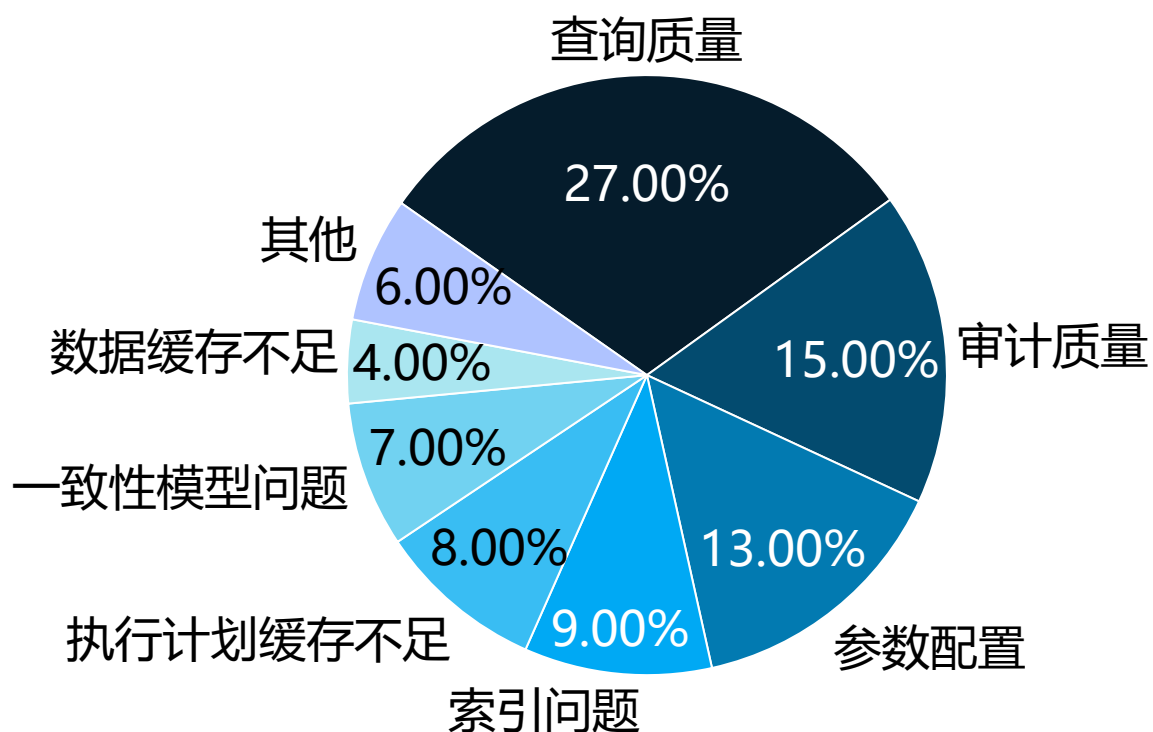


异构硬件，加速执行

智能数据库系统 (AI4DB)

数据库性能瓶颈的成因复杂，性能优化是一项高门槛、高投入的工作，资深DBA也需要花费大量时间理解当前情况进行手动调整

数据库性能问题成因调查



解决方案

粗放管理，不断扩容高性能服务器

高价雇佣资深DBA，利用经验手动控制

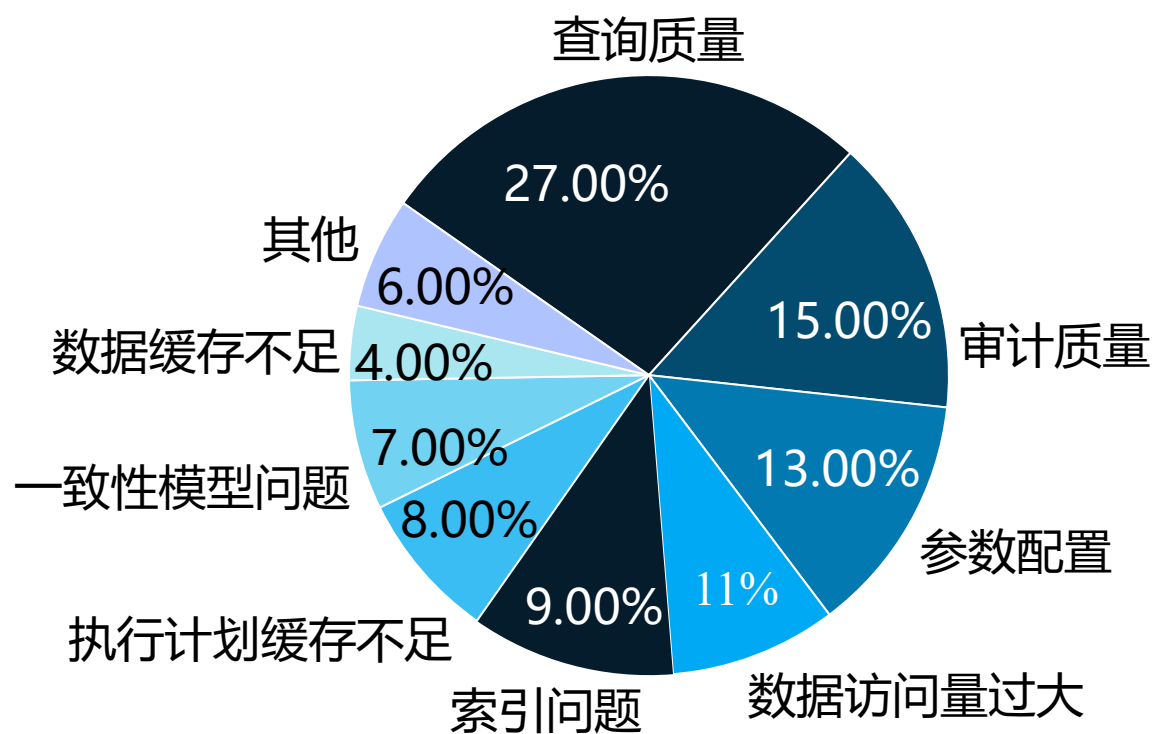
特定场景的启发式算法优化

基于机器学习和专家经验的数据库自治服务

智能数据库系统 (AI4DB)

数据库性能瓶颈的成因复杂，性能优化是一项高门槛、高投入的工作，资深DBA也需要花费大量时间理解当前情况进行手动调整

数据库性能问题成因调查



解决方案

粗放管理，不断扩容高性能服务器

高价雇佣资深DBA，利用经验手动控制

特定场景的启发式算法优化

基于机器学习和专家经验的数据库自治服务

智能数据库系统 (AI4DB)

□ 智能数据库内部优化

- 逻辑查询重写
- 执行代价估计
- 物理计划选择

□ 智能数据库执行器

- 并行性能预测

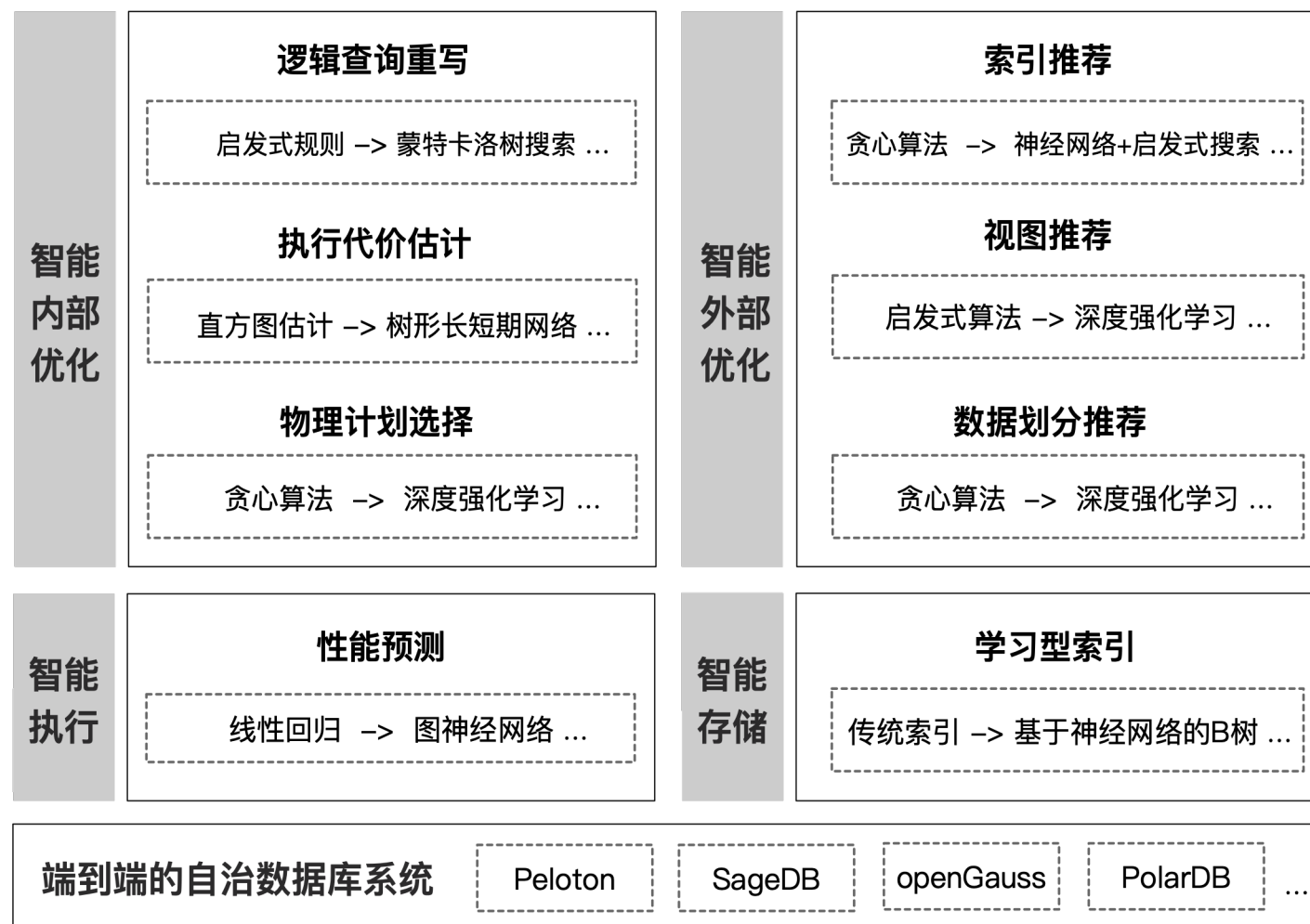
□ 智能数据存储引擎

- 学习型索引

□ 智能数据库外部优化

- 智能索引推荐
- 智能视图推荐
- 智能数据划分

□ 自治数据库系统



硬件—数据库技术发展的动力之一

□硬件技术的发展

- 硬件技术决定了数据存取、查询处理等性能的上限
- 新硬件产品的出现提高数据库的性能上限
 - 多核处理器（CPU）
 - 协处理器（GPU、FPGA）
 - 非易失性存储（NVM）
 - 高速网络（RDMA）
- 数据库软件需要针对新硬件的特点设计架构和算法
 - 提高软件硬件契合度，尽可能“压榨”硬件的性能

区块链与数据库

□ 区块链是一种安全共享的去中心化的数据账本

- 因其分布式、去中心和信息不可篡改等特性越来越受到关注。
- 从某种角度来说，区块链是一个去中心化的数据库，但是其对数据的查找和数据格式化处理方面有天生的不足。

□ 数据库技术提高区块链的性能

- 在以区块链作为数据存储层的基础上，研究如何将数据库技术与区块链结合起来，为区块链提供一个数据库层以加速数据的查询效率、提高区块链作为数据库的可用性也可能是未来的发展趋势。

□ 基于区块链技术的数据库

- 基于区块链的不可变特性来研发数据不可改变的数据库用于特定的用途也是一个可以探索的方向。

区块链与数据库（续）

对比点		设计选择	原因
共识协议	区块链	拜占庭故障容忍	互不信任的运行环境
	数据库	宕机故障容忍	高可用的运行要求
共识层位置	区块链	顶层	去中心化，防单点
	数据库	存储层之上	数据完备性
事务层	区块链	顺序执行	易于理解、易用
	数据库	并发执行	高吞吐
存储	区块链	历史（账本）+状态	溯源，解决争议
	数据库	状态	节约存储
索引	区块链	Merkle Tree等	篡改可见性
	数据库	B树	硬件友好
Sharding	区块链	基于节点的分片	防止恶意分区
	数据库	基于服务的分片	负载均衡

目录

- 数据库系统概述
- 数据库发展简史
- 数据库系统的应用和展望
- 本章小结

本章小结

- ❑ 数据库技术是信息技术领域的核心技术之一，几乎所有的信息系统都需要使用数据库来组织、存储、操纵和管理业务数据。数据库是数据管理的有效手段之一，对数据库的研究也是现代计算机学科的重要分支和方向。
- ❑ 本章通过阐述数据、数据库、数据库管理系统和数据库系统的概念，抽丝剥茧地进行数据库系统的概述。本章介绍了数据库管理系统的查询接口层、查询处理层、查询执行层、存储管理层和管理组件五个模块组成，并在相应的模块介绍了一些数据库的重要概念，如事务管理和索引管理等。
- ❑ 本章介绍了数据库的发展简史，阐述了新技术和新需求对数据库演化的影响。
- ❑ 本章的最后还总结了基于数据库的典型应用系统，并展望了人工智能、区块链和新硬件等技术对数据库发展的影响。